Exp-04: When Cache Optimizations Make the Processor Slower

Deetya Ashish Mehta
Roll No: CS23I1032

September 7, 2026

Abstract

This report studies four cache optimizations on a five-stage synthetic data-processing pipeline (ingestion, feature extraction, feature aggregation, lookup/scoring, result generation): a victim cache, three prefetch mechanisms (next-line, stride, stream), and a non-blocking cache with MSHRs. The cache hierarchy is a simplified model of an Intel Xeon Platinum 8480+ (Sapphire Rapids) core: 48 KB/12-way L1D, 2 MB/16-way L2, 15 MiB/15-way aggregate LLC, 64 B lines. Every table below is generated directly from a run of the accompanying simulator and \inputed verbatim, so the numbers in the text match the code exactly. Four results carry the report's main point, that the right optimization depends on the workload and that fewer misses does not mean a faster processor. A distance-1 stride prefetcher cuts the L1 miss rate from 57.4% to 23.2% but makes execution *slower*, because its prefetches arrive with 0% timeliness and the extra traffic evicts data still in use. A victim cache that does almost nothing on the full pipeline (0.008% faster) removes 61.6% of the cycles in a purpose-built conflict micro-benchmark, with a sharp cutoff at exactly (N − ways) entries. Under high interleave pressure, a stream prefetcher drops the overall L1 miss rate from 99.6% to 1.0% while leaving the specific structure that is thrashing completely unaffected. And in a dedicated MSHR timing model, extra outstanding-miss slots buy nothing for a dependent pointer-chase but cut an 8-stream independent workload's time 8×, plateauing exactly at MSHR = 8.

Contents

1 Introduction

Modern processors already ship with a carefully tuned cache hierarchy. The question this assignment asks is not whether more cache machinery reduces misses, since it almost always can, but whether it reduces *execution time*, and under what conditions it stops helping or starts hurting. To study that question honestly requires a workload realistic enough to exercise several access patterns at once (streaming, strided, pointer-chasing, mixed random/regular), and a simulator detailed enough to separate "fewer misses" from "faster execution" as genuinely different outcomes.

Section 2 describes the simplified processor model. Section 3 describes the five-stage workload built for this study. Sections 4–9 present the seven experiment groups: baseline characterization, victim cache, prefetch mechanisms, prefetch distance, cache pollution and thrashing, and blocking vs. non-blocking caches with MSHRs. Section 10 combines the findings into a final recommended configuration, and Section 11 checklists every concept the assignment asks to see demonstrated against the section that demonstrates it.

2 Processor Model and System Parameters

Table 1 lists the simplified 8-core-subset model used throughout. Capacities and line size follow the assignment exactly (L1 48 KB, L2 2 MB, LLC 1.875 MB/core × 8 = 15 MiB aggregate, 64 B lines); associativity, per-level latency, and DRAM latency are reasoned choices for parameters the assignment leaves open, and are meant as an architectural reference rather than a bit-for-bit reproduction of the real chip, which the assignment states is acceptable.

Level	Parameters	Rationale
L1D	48 KB, 12-way, 64 sets, 5 cyc	Golden Cove L1D is 48 KB/12-way
L2	2 MB, 16-way, 2048 sets, 16 cyc	assignment-given capacity
LLC	15 MiB (aggregate), 15-way, 16384 sets, 60 cyc	8×1.875 MB/core
DRAM	200 cyc	illustrative SPR/DDR5-class latency
Victim cache	fully associative, +3 cyc over L1 hit	small, parallel lookup
Line size	64 B (all levels)	assignment-given

Table 1: Simplified processor/cache model used for every experiment.

All three set counts (64, 2048, 16384) come out to exact powers of two given these capacities, associativities, and line size. The 15-way LLC looks unusual on paper, but it is exactly what 15 MiB/64 B = 245760 lines factors into with $16384 = 2^{14}$ sets, and real Intel LLC slices are often non-power-of-two-way for the same reason.

Timing model. Every experiment except Section 9 uses a cumulative, serial latency model: an access that misses in L1 and hits in L2 costs $L1_{lat} + L2_{lat}$; a full miss costs $L1_{lat} + L2_{lat} + LLC_{lat} + DRAM_{lat}$. This is a single-outstanding-request (blocking) model, adequate for every question about miss rates, pollution, and prefetch effectiveness, but blind to memory-level parallelism by construction: run through it, a dependent pointer-chase and eight independent streams of the same size come out to the identical total clock, since the model has no notion of two misses ever being outstanding at once. That is exactly why Section 9 uses a separate, request-level timing model instead.

Prefetched lines are inserted into the cache at issue time, so an early or wasted prefetch correctly participates in LRU/eviction bookkeeping the moment it is fetched. Each prefetched line also carries a predicted arrival clock; a demand access that arrives before that clock still pays the remaining wait, which is what makes a "late" prefetch measurable. Every prefetcher here targets L1 directly (prefetch requests walk the full hierarchy and fill up to L1), so L1 pollution is visible rather than hidden behind L2/LLC.

3 Workload

A five-stage pipeline is implemented in `src/workload.py` as an address-stream generator, in the same style as the Exp-01/Exp-03 simulators: no data values are modeled, only the addresses a real implementation would touch. That is enough, since every phenomenon under study here is about where and when memory is accessed, not what arithmetic runs on it.

1. **Data Ingestion.** A sequential scan of an 80,000-record array (128 B/record, 2 cache lines each), ≈9.8 MB and touched once per record, so there is no temporal reuse in the record stream itself. A small 4 KB metadata block is re-touched twice per record, standing in for reused processing parameters.

2. **Feature Extraction.** Re-reads the same record array at a configurable record-level stride, from contiguous up to arbitrarily large, touching a fixed 60,000 records regardless of stride so useful work stays constant while only the address pattern changes. Forward and backward traversal are both supported.

3. **Feature Aggregation.** A small 512-entry (4 KB) accumulator, updated once every 4 input records. This is the workload's "small, frequently reused state" structure, central to Section 8.

4. **Lookup and Scoring.** A 200,000-entry (≈6.1 MB) index accessed with a 60/40 mix of pseudo-random indices and a periodic, partially-predictable regular component.

5. **Result Generation.** Two variants of equal footprint (≈2.56 MB, 40,000 nodes): a dependent pointer-chase (visitation order is a precomputed random cyclic permutation, the standard way to model pointer-chasing without simulating memory contents) and $K = 8$ independent chains, round-robin interleaved. Independence here means stream k never waits on stream j's outstanding request, but each stream is still internally a dependent chain, which is what gives memory-level parallelism a finite bound (K) instead of an unrealistic unbounded one.

The full pipeline runs stages 1–5 back to back through one simulator instance, without clearing cache state between stages. Section 4 shows this directly: Extraction's LLC miss rate is 0.00% because it re-reads data Ingestion just streamed through the LLC.

4 Experiment A: Baseline Characterization

Table 2 and Figure 1 give the per-stage miss rate and AMAT of the unmodified pipeline: no victim cache, no prefetching.

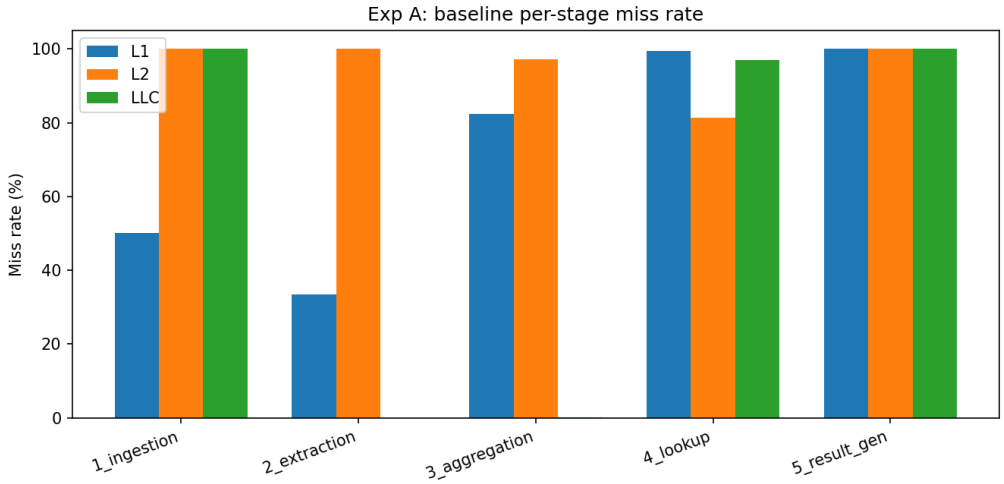

Figure 1: Baseline per-stage miss rate by cache level.

Stage	Accesses	L1 Miss%	L2 Miss%	LLC Miss%	DRAM Acc.	AMAT
1_ingestion	320,000	50.00	100.00	100.00	160,002	143.00
2_extraction	180,000	33.33	100.00	0.00	0	30.33
3_aggregation	100,000	82.41	97.15	0.08	64	66.35
4_lookup	60,000	99.40	81.35	96.91	47,020	226.16
5_result_gen	40,000	100.00	100.00	100.00	40,000	281.00

Table 2: Exp A: baseline per-stage characterization (no victim cache, no prefetching).

Stage segment	Accesses	L1 Miss%	AMAT
1_ingestion_cold_0-20pct	64,000	50.00	143.01
1_ingestion_steady_20-100pct	256,000	50.00	143.00
2_extraction_cold_0-20pct	36,000	33.33	30.33
2_extraction_steady_20-100pct	144,000	33.33	30.33
3_aggregation_cold_0-20pct	20,000	82.59	67.05
3_aggregation_steady_20-100pct	80,000	82.37	66.18
4_lookup_cold_0-20pct	12,000	99.42	269.19
4_lookup_steady_20-100pct	48,000	99.40	215.40
5_result_gen_cold_0-20pct	8,000	100.00	281.00
5_result_gen_steady_20-100pct	32,000	100.00	281.00

Table 3: Exp A: first 20% (cold) vs. remaining 80% (steady-state) of each stage.

Ingestion is 100% miss at L2 and LLC, and 50% miss at L1, matching the two per-record lines missing and the two metadata touches after the first hitting: a clean capacity-miss stream. Extraction's LLC miss rate is 0.00%, since it re-reads the same 9.8 MB array Ingestion just finished streaming through and the 15 MiB LLC can still hold essentially all of it, which is direct evidence of state carried across stages. Aggregation's L1 miss rate (82.4%) is higher than Extraction's despite reusing the same record array, because it interleaves that scan with writes to `agg_state`, which foreshadows Section 8. Lookup is the worst stage by AMAT (226 cycles): a ≈6.1 MB structure touched for the first time, 60% randomly, so even the LLC only catches 3.1% of the L2-missing traffic. Result Generation is a clean 100% miss at every level by design.

Table 3 splits each stage into its first 20% ("cold") and remaining 80% ("steady-state"). Ingestion and Extraction show a small early-stage AMAT bump while the metadata block and

front of the record array are still filling in; Lookup and Result Generation show almost no cold/steady difference, since neither working set is ever revisited within one pass.

5 Experiment B: Victim Cache

Table 4 sweeps victim-cache size against the full pipeline.

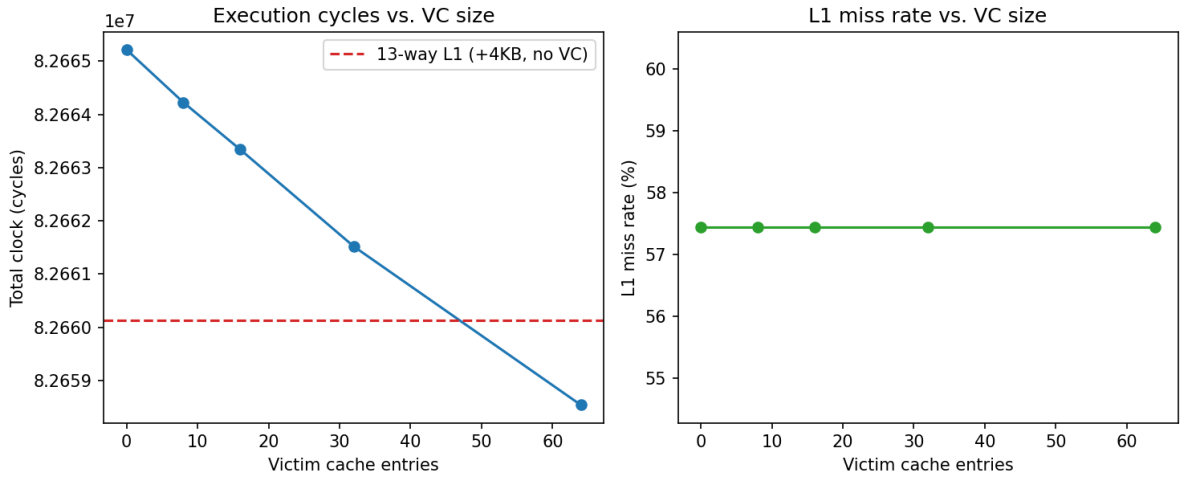

Figure 2: Full-pipeline victim-cache sweep: execution cycles and L1 miss rate vs. victim-cache size, with the equal-extra-storage 13-way L1 alternative shown as a reference line.

VC entries	Extra bytes	Clock (cyc)	L1 Miss%	VC hits	AMAT
0	0	82,665,212	57.44	0	118.09
8	512	82,664,224	57.44	76	118.09
16	1,024	82,663,340	57.44	144	118.09
32	2,048	82,661,520	57.44	284	118.09
64	4,096	82,658,543	57.44	513	118.08
13-way L1 (+4KB, no VC)	4,096	82,660,140	57.39	–	118.09

Table 4: Exp B: victim-cache size sweep vs. an equal-extra-storage associativity increase.

On the full pipeline a victim cache is essentially invisible: even 64 entries only shave 0.008% off total cycles, and move the L1 miss rate by less than 0.01 points. This is a real result, not a bug. The full pipeline's misses are overwhelmingly cold or capacity misses on datasets far larger than any cache (Section 4), and a victim cache targets *conflict* misses specifically, a handful of addresses repeatedly aliasing to the same set despite the working set nominally fitting. A one-pass streaming scan almost never produces that signature. The equal-extra-storage alternative, an L1 widened to 13-way (the same +4 KB as the 64-entry victim cache), does marginally better than the victim cache here (82,660,140 vs. 82,658,543 cycles): with almost no conflict-miss opportunity to recover, spending the extra 4 KB as unconditional L1 capacity is at least as good as reserving it for victim-only overflow.

To show the behavior a victim cache is actually built for, a supplementary micro-benchmark isolates it directly: 15 addresses, all mapping to the same 12-way L1 set, accessed round-robin for 3000 rounds, a classic pathological pattern in which every access misses once the address count exceeds the set's associativity.

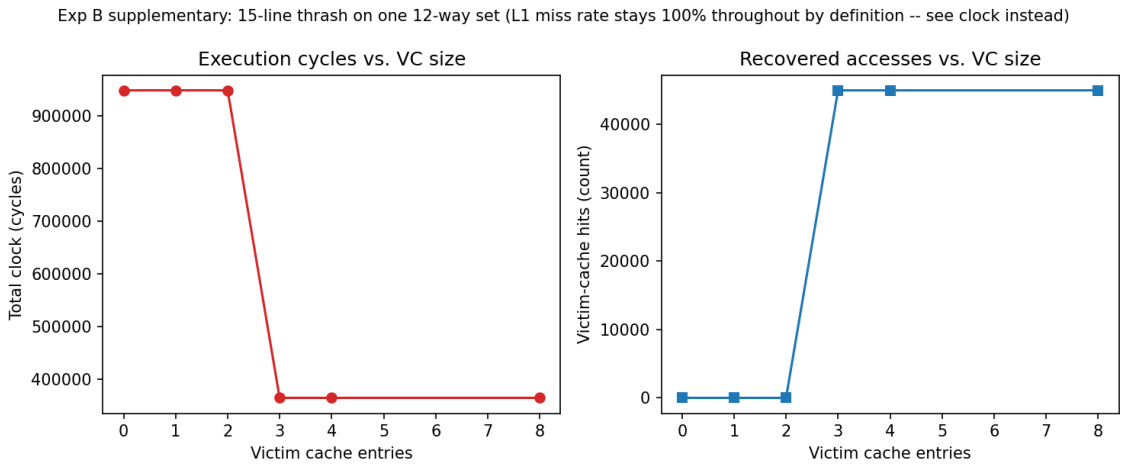

Figure 3: Conflict micro-benchmark: execution cycles and recovered (victim-cache-hit) accesses vs. victim-cache size. L1 miss rate stays flat at 100% throughout by definition, since a victim-cache hit is still an L1 miss, even as cycles fall sharply.

VC entries	L1 Miss%	Clock (cyc)	VC hits
0	100.00	948,900	0
1	100.00	948,900	0
2	100.00	948,900	0
3	100.00	364,095	44,985
4	100.00	364,095	44,985
8	100.00	364,095	44,985

Table 5: Exp B supplementary: 15 addresses round-robin thrashing one 12-way L1 set.

Here the threshold is sharp: 0, 1, and 2 victim entries give zero improvement (clock stays at 948,900 cycles), but at exactly VC = 3, which is $N - \text{ways} = 15 - 12$, clock drops to 364,095 cycles (a 61.6% reduction) and 44,985 accesses become victim-cache hits. VC = 4 and VC = 8 give no further improvement at all: once the victim cache is exactly large enough to hold the overflow the primary cache cannot, more capacity is wasted. This is the answer the full-pipeline sweep could not give: a victim cache recovers data when a small number of addresses thrash one set, and its useful capacity stops exactly at (contending addresses − associativity).

Worth noting: the micro-benchmark's L1 miss rate reads 100% in every row of the underlying data, even where clock drops 61.6%, because a victim-cache hit is still an L1 miss by definition. A raw miss-rate number at one level can look unchanged while execution time moves by more than half.

6 Experiment C: Prefetch Mechanisms and Effectiveness

Three prefetchers, implemented in `src/cache_core.py`, are applied to the complete workload rather than hand-picked to the stage each suits best: **next-line** (stateless, on every L1 miss to line L prefetch $L + 1$), **stride** (per-stream two-delta detector, one prefetch issued `distance` strides ahead once confirmed), and **stream** (per-stream run-ahead engine, requiring two consecutive matching deltas to confirm, then maintaining a `distance`-line run-ahead window). "Per-stream" state is keyed on the workload's logical stream tags (`ingest_records`, `extract`, `lookup`, ...), the simulator-level stand-in for the per-PC stride tables real hardware uses.

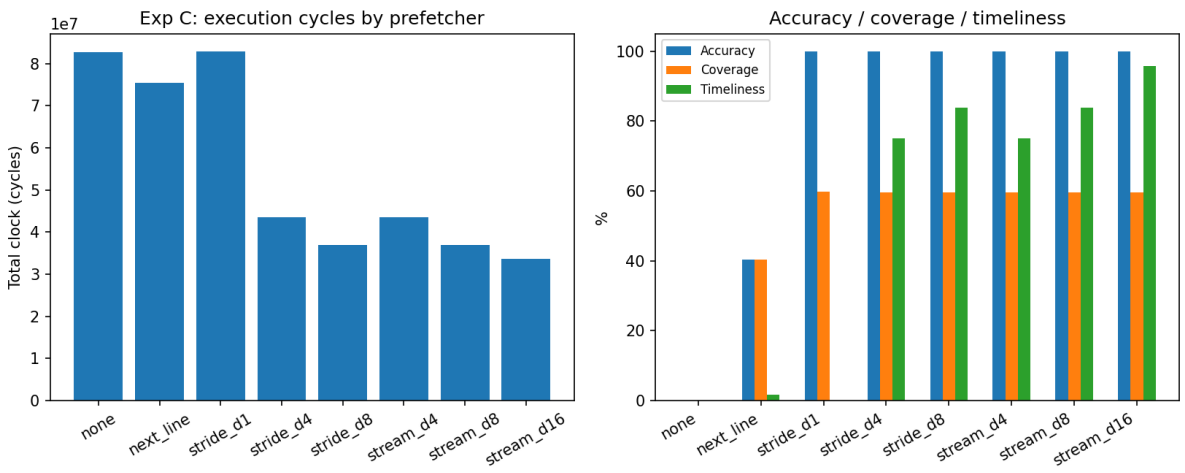

Figure 4: Left: total execution cycles by prefetcher. Right: accuracy, coverage, and timeliness (%) by prefetcher.

Prefetcher	Clock (cyc)	L1 Miss%	Accuracy%	Coverage%	Timeliness%	Issued
none	82,665,212	57.44	0.0	0.0	0.0	0
next_line	75,510,773	34.57	40.4	40.4	1.6	402,223
stride_d1	82,920,183	23.15	100.0	59.7	0.0	240,060
stride_d4	43,439,008	23.16	100.0	59.7	75.0	240,105
stride_d8	36,863,516	23.16	100.0	59.7	83.9	240,107
stream_d4	43,444,992	23.16	100.0	59.7	75.0	239,998
stream_d8	36,867,718	23.16	100.0	59.7	83.9	240,006
stream_d16	33,689,342	23.17	100.0	59.7	95.8	240,022

Table 6: Exp C: prefetch mechanism comparison, full pipeline.

Accuracy, coverage, and timeliness are computed as specified: accuracy is useful prefetches over issued prefetches; coverage is useful prefetches over baseline (no-prefetch) demand misses; timeliness is the fraction of useful prefetches that had already arrived by the time they were referenced. Prefetch traffic (DRAM/cache accesses generated by prefetch requests) is counted completely separately from demand traffic.

The central finding is in the `stride_d1` row: 100.0% accuracy, L1 miss rate more than halved (57.4% $\rightarrow$ 23.2%), yet total cycles rise slightly, from 82,665,212 to 82,920,183. Every issued prefetch eventually gets used, but timeliness is 0.0%: at distance 1 the prefetch target is only one stride ahead of the access that triggered it, so the demand stream catches up to (and still waits out) essentially every in-flight prefetch, while the extra traffic still evicts data that was genuinely still in use. High accuracy and a large drop in the raw miss count provide no performance benefit here, because a prefetch only pays off by hiding latency behind other work, and distance 1 leaves nothing to hide it behind.

`next_line` shows the opposite failure mode: accuracy only 40.4% and timeliness 1.6%, since it fires unconditionally regardless of whether the current stream is even sequential. It does reasonably well on Ingestion's two-line-per-record scan (field 0 and field 8 sit on adjacent lines), but is essentially blind guessing on Lookup's mostly-random index stream. It still nets a win overall (82.67M $\rightarrow$ 75.51M cycles), purely because Ingestion's regularity outweighs its failures elsewhere, but it is clearly the least reliable of the three mechanisms.

7 Experiment D: Prefetch Distance and Unintended Effects

Table 7 and Figure 5 sweep prefetch distance for stride and stream across the full pipeline, including the no-prefetch reference point (distance 0).

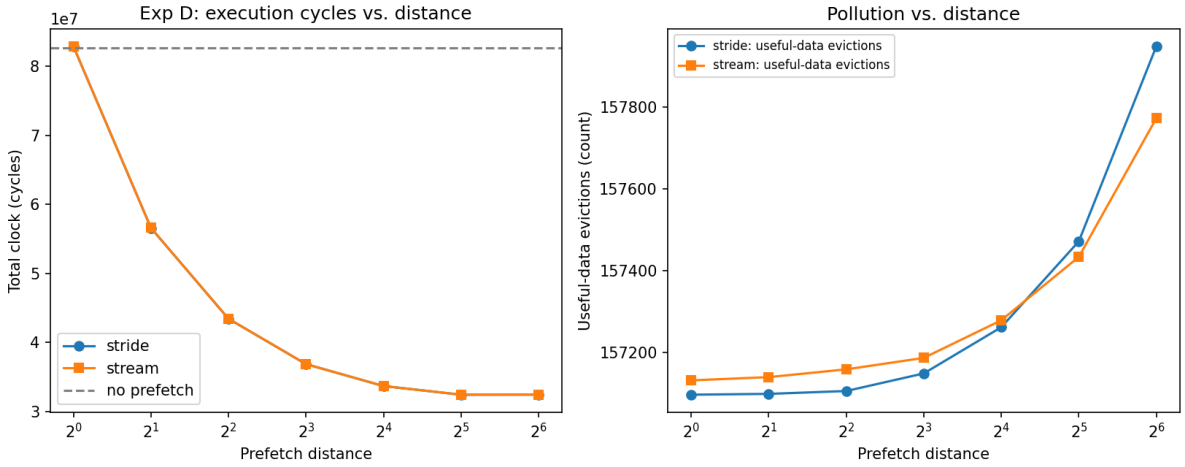

Figure 5: Left: execution cycles vs. prefetch distance (log scale). Right: useful-data evictions (pollution) vs. distance.

Kind	Distance	Clock (cyc)	L1 Miss%	Wasted-evicted PF	Useful-data evictions
none	0	82,665,212	57.44	0	765,916
stride	1	82,920,183	23.15	248,860	157,097
stride	2	56,600,572	23.15	248,909	157,099
stride	4	43,439,008	23.16	248,973	157,106
stride	8	36,863,516	23.16	248,985	157,149
stride	16	33,685,428	23.18	249,027	157,262
stride	32	32,432,768	23.20	249,061	157,472
stride	64	32,449,932	23.26	249,080	157,949
stream	1	82,922,825	23.15	248,761	157,132
stream	2	56,603,970	23.15	248,763	157,140
stream	4	43,444,992	23.16	248,771	157,159
stream	8	36,867,718	23.16	248,791	157,187
stream	16	33,689,342	23.17	248,826	157,279
stream	32	32,431,948	23.20	248,894	157,434
stream	64	32,437,372	23.25	248,969	157,773

Table 7: Exp D: prefetch distance sweep, full pipeline (distance 0 = no prefetch).

For both mechanisms, distance 1 is a net loss against no prefetching; distance 2 already recovers to a large net win (56.6M cycles); cycles keep falling through distance 32 (32.43M cycles, timeliness 95–96%) as prefetches increasingly arrive with time to hide the miss latency; and distance 64 is very slightly worse than distance 32 for both mechanisms (32.45M vs. 32.43M stride, 32.44M vs. 32.43M stream), even though its raw miss count is marginally lower still. At that point the run-ahead window is far enough ahead of the demand stream that some of what it fetches is evicted before use, adding traffic without adding benefit. Together these three distances trace the "too late / just right / too aggressive" arc inside one mechanism.

A demand miss costs the full serial latency (≈281 cycles here) whether or not a prefetch was issued for that line. A prefetch only turns that cost into a win if it completes before the demand needs it and does not itself displace data that would otherwise still be there. Reducing

the demand-miss count is necessary but not sufficient for reducing execution time: accuracy alone only says a prefetch was eventually useful, not that it was useful in time, and coverage says nothing about what it evicted along the way.

8 Experiment E: Cache Pollution and Thrashing

Sections 4–7 show pollution as a side effect; this experiment builds it on purpose. Ingestion, Extraction, and Aggregation run genuinely interleaved at record granularity rather than stage by stage: for every record, two ingestion lines are touched, then `pressure` fresh, never-before-touched filler lines are touched (standing in for continuously arriving input), and every 4th record also touches the small, hot `agg_state` array. `pressure` directly and monotonically controls how much foreign traffic separates one touch of the hot state from the next. An earlier version of this experiment drove pressure through a modular record stride instead of a never-repeating filler region, and produced a non-monotonic curve caused by the stride sharing a factor with the array length – a reminder of how easily a synthetic "pressure" generator can build in unintended extra reuse.

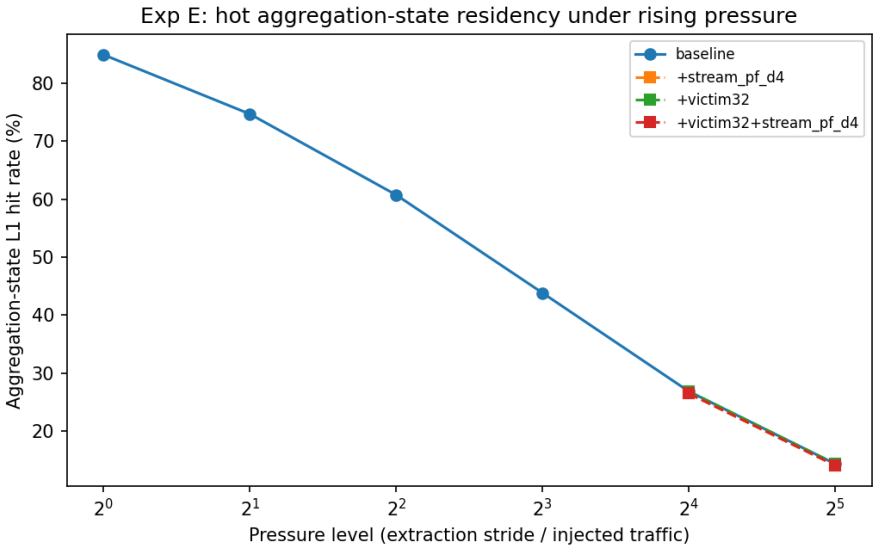

Figure 6: Aggregation-state L1 hit rate vs. pressure, baseline and with victim-cache / stream-prefetcher mitigation at the two highest pressure levels.

Pressure level	Aggregation-state L1 hit%	Overall L1 Miss%
1	84.92	93.47
2	74.69	95.61
4	60.73	97.57
8	43.82	98.93
16	26.82	99.63
32	14.32	99.90

Table 8: Exp E: baseline aggregation-state residency under rising interleave pressure.

Pressure	Mitigation	Aggregation-state L1 hit%	Overall L1 Miss%
16	+victim32	26.82	99.63
16	+stream_pf_d4	26.46	1.01
16	+victim32+stream_pf_d4	26.46	1.01
32	+victim32	14.32	99.90
32	+stream_pf_d4	14.06	0.63
32	+victim32+stream_pf_d4	14.06	0.63

Table 9: Exp E: does the victim cache and/or prefetching alleviate high-pressure thrashing?

The aggregation state's own L1 hit rate falls, monotonically, from 84.9% at the lowest pressure to 14.3% at the highest, while the workload's overall L1 miss rate keeps climbing toward ≈100% throughout regardless of pressure, since it is dominated by the ever-growing, inherently-cold filler stream. This is the distinction between a large working set and repeatedly destroyed cache residency: the aggregation state itself is tiny (512 entries, 4 KB, it would trivially fit in L1 alone) and is never too large for the cache on its own. What destroys its hit rate is the volume of unrelated traffic injected between two of its own touches, which at high pressure exceeds the L1's associativity in nearly every set before the state is next needed.

Do the other mechanisms help or intensify this? They answer differently. A 32-entry victim cache changes nothing: at pressure 16 and 32 both the aggregation-state hit rate and overall miss rate are identical to baseline. A continuous flood of always-cold filler lines touching essentially every set overwhelms 32 entries; this is capacity pressure spread broadly across the cache, not the concentrated same-set conflict pattern Section 5's micro-benchmark isolates. A distance-4 stream prefetcher does the opposite: overall L1 miss rate collapses from 99.6% to 1.0% at pressure 16 (99.9% → 0.63% at pressure 32), since the filler stream is purely sequential, exactly what a stream prefetcher is built for. But the aggregation state's own hit rate is essentially unchanged (26.82% → 26.46% at pressure 16, if anything marginally worse, since the prefetcher adds still more traffic competing for the same sets). A mechanism can be a large net win for a workload as a whole while being irrelevant, or mildly counter-productive, for one specific structure inside it.

9 Experiment F: Blocking vs. Non-Blocking Cache and MSHRs

The blocking clock used everywhere above cannot show memory-level parallelism by construction: it sums per-access latencies in program order and has no notion of two misses being outstanding at once, so the dependent-chain and independent-streams variants of Result Generation come out to the same total clock through it. `src/mlp_model.py` reclassifies the same access stream (hit/miss and per-access service latency, taken from one blocking-model pass) through an explicit MSHR-timing model instead: a request needs a free MSHR slot to start service, and stalls if all `mshr_count` slots are busy; within one logical stream, the next access cannot issue before the previous access on that same stream completes. That last rule is what makes "independent streams" independent of *each other* while still leaving each stream's own requests properly serialized, so achievable parallelism is bounded by $K = 8$ rather than unrealistically unbounded.

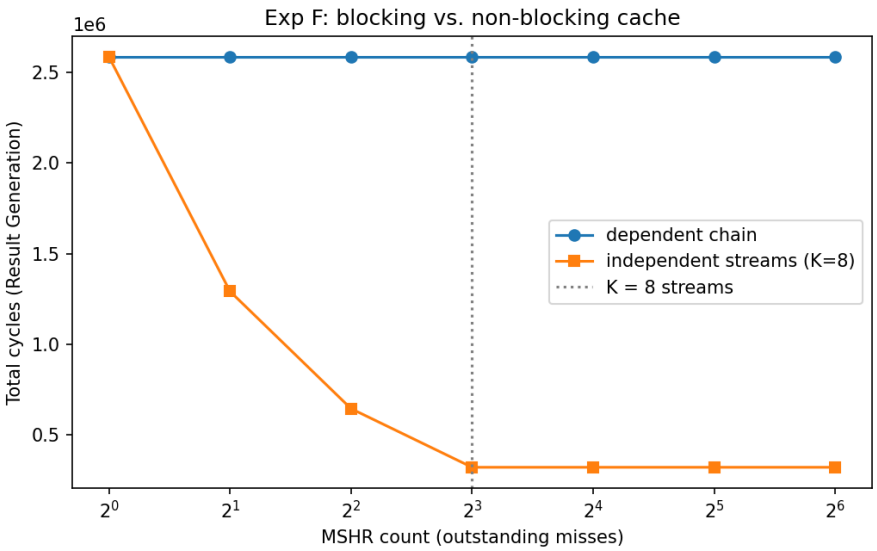

Figure 7: Total Result-Generation execution cycles vs. MSHR count (log-scale x-axis), dependent chain vs. 8 independent streams.

MSHRs	Dependent chain (cyc)	Independent streams (cyc)
1	2,584,000	2,584,000
2	2,584,000	1,292,001
4	2,584,000	646,003
8	2,584,000	323,007
16	2,584,000	323,007
32	2,584,000	323,007
64	2,584,000	323,007

Table 10: Exp F: total Result-Generation execution cycles vs. MSHR count.

The dependent chain is flat at 2,584,000 cycles from MSHR = 1 through MSHR = 64: extra outstanding-miss slots cannot help a request stream that never has more than one independent request in flight, since the next address is unknown until the current one returns. The independent-streams variant instead falls by almost exactly half with every doubling of MSHR count (2,584,000 → 1,292,001 → 646,003 → 323,007 for MSHR = 1,2,4,8), textbook memory-latency hiding, and then plateaus exactly at MSHR = 8, matching $K = 8$ streams precisely: MSHR = 16, 32, and 64 all give the identical 323,007 cycles, since at most 8 requests can ever be simultaneously ready to issue.

MSHRs (miss status holding registers) are what make a non-blocking cache non-blocking: each tracks one in-flight miss so the pipeline can keep accepting new requests instead of stalling behind the first one. Outstanding misses are exactly the requests those MSHRs track; a request that finds none free must itself stall, and MSHR = 1 is simply the blocking case, confirmed above since the independent-streams row at MSHR = 1 matches the dependent chain exactly. Memory-level parallelism is the amount of concurrent, mutually-independent outstanding-miss activity a request stream can offer; memory-latency hiding is what non-blocking hardware does with that opportunity, overlapping one miss's wait behind another's. Extra MSHRs help the independent-streams workload far more than the dependent one because only the former has parallelism to exploit, and they stop helping the moment their count reaches the workload's own ceiling on simultaneously-ready requests, a property of the algorithm that no amount of extra buffering can raise.

10 Final Performance Analysis and Recommendation

Table 11 compares four full-pipeline configurations: baseline; victim-cache-only (64 entries, best from Section 5); prefetch-only (stream at distance 16, best single mechanism from Sections 6–7); and both combined.

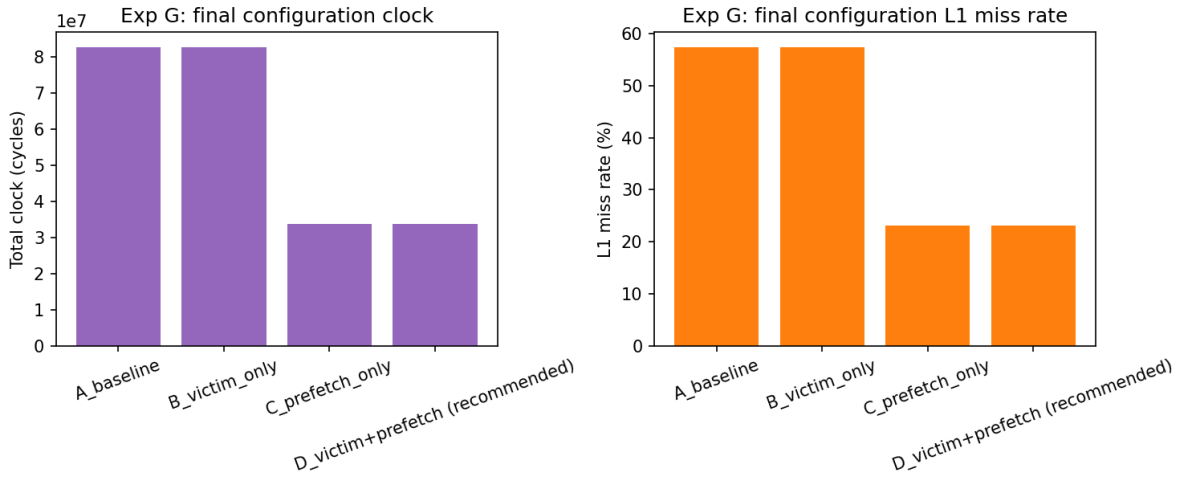

Figure 8: Final configuration comparison: execution cycles (left) and L1 miss rate (right).

Configuration	Clock (cyc)	AMAT	L1 Miss%	DRAM Acc.
A_baseline	82,665,212	118.09	57.44	247,086
B_victim_only	82,658,543	118.08	57.44	247,086
C_prefetch_only	33,689,342	48.13	23.17	87,090
D_victim+prefetch (recommended)	33,682,036	48.12	23.17	87,090

Table 11: Exp G: final configuration comparison (selected prefetcher: stream_d16, selected victim-cache size: 64 entries).

Victim cache alone moves 82,665,212 → 82,658,543 cycles, a 0.008% improvement, confirming Section 5: this workload does not offer the victim cache's target pattern at meaningful scale. Prefetch alone moves 82,665,212 → 33,689,342 cycles, a 59.2% reduction and by far the dominant lever available, since Ingestion, Extraction, and Lookup all contain enough sequential or strided structure for a well-tuned stream prefetcher to exploit, and distance 16 sits in the zone that arrives with time to hide the miss without meaningfully polluting. Both combined moves 33,689,342 → 33,682,036 cycles, an additional 0.02% on top of prefetching alone; the victim cache's marginal contribution barely changes once prefetching is active, for the same reason it contributed almost nothing on its own.

Recommendation. Given the assignment's hardware/complexity budget, the victim cache is not recommended for this workload: real design cost (fully-associative tag comparators, a swap path between L1 and the victim structure) for a measured 0.008–0.02% return in every configuration tested. It would be worth it only if a production version of this pipeline were expected to hit the same-set-conflict pattern Section 5's micro-benchmark isolates, for example a hash table or index with a poor address-to-set distribution, a case this workload as built does not trigger. A stream prefetcher at a moderate distance (8–16) is clearly worth it: it captures most of the achievable improvement, sits past the "too late" zone with only mild pollution cost, and (Section 8) stays a strong net win even under elevated interleave pressure. A non-blocking cache with a modest MSHR budget (8–16) is recommended for the Result-Generation stage

and any other stage with genuine independent parallelism (Lookup's regular sub-component is a second candidate): Section 9 shows this captures essentially all of the available memory-level-parallelism benefit for a workload whose real independent-stream count is 8, with no return past that point, and correctly no return at all for the pipeline's dependent-chain portions, where no amount of buffering substitutes for a sequential dependency the algorithm itself imposes.

11 Required-Observations Checklist

Concept	Evidence
Victim Cache	§5, Tables 4–5
Next-Line Prefetching	§6, Table 6 (`next_line` row)
Stride Prefetching	§6, Table 6 (`stride_*` rows)
Stream Prefetching	§6, Table 6 (`stream_*` rows)
Prefetch Distance	§7, Table 7, Fig. 5
Prefetch Accuracy	§6, Table 6 col. 3
Prefetch Coverage	§6, Table 6 col. 4
Prefetch Timeliness	§6, Table 6 col. 5; `stride_d1` case
Prefetch Traffic	§6/7, Table 6/7 issued/DRAM-prefetch cols
Cache Pollution	§6 (`stride_d1`), §8
Cache Thrashing	§8, Table 8; §5 conflict micro-bench
Useful-Data Eviction	§7, `useful_data_evictions` in Table 7
Memory-Bandwidth Pressure	§8 (filler traffic vs. agg. state)
Prefetch Interference	§8 (stream prefetch vs. agg. state hit rate)
Blocking Cache	§9, MSHR = 1 row of Table 10
Non-Blocking Cache	§9, MSHR > 1 rows of Table 10
MSHRs	§9
Outstanding Misses	§9
Memory-Level Parallelism	§9, independent-streams curve, Fig. 7
Memory-Latency Hiding	§9, halving-per-MSHR-doubling result

Table 12: Where each concept required by the assignment is demonstrated with measured data.

12 Limitations

- Latencies (Table 1) are illustrative round numbers for a Sapphire-Rapids-class hierarchy, not measured or vendor-published values, which the assignment explicitly allows.
- The blocking timing model used everywhere except Section 9 assumes one outstanding request at a time; absolute cycle counts there should be read as relative comparisons under one consistent assumption, not as absolute predicted latencies.
- No dirty-bit/write-back state is modeled, matching the Exp-01/Exp-03 convention; a line evicted from the victim cache is simply dropped rather than written back, assuming L2/LLC already hold a copy.
- The 8-core / 15 MiB-aggregate-LLC model is exercised as a single-stream simulation against that aggregate hierarchy, per the assignment's framing of the workload as one large-scale pipeline; true multi-core contention for the shared LLC is out of scope here.
- Prefetcher per-stream state is keyed on the workload generator's logical stream tags, a simulator-level stand-in for per-PC stride tables in real hardware, not on true instruction addresses.

A Source Code

The complete, runnable source is submitted alongside this report (four files: `src/cache_core.py`, `src/workload.py`, `src/mlp_model.py`, `run_experiments.py`). Running `python3 run_experiments.py` regenerates every table, CSV, JSON file, and plot referenced above from scratch (fixed random seed, so the numbers are exactly reproducible). All four are reproduced in full below.

Listing 1: `src/cache_core.py` – generic set-associative cache, victim cache, hierarchy simulator, and prefetcher classes

```
"""
Exp-04: When Cache Optimizations Make the Processor Slower.

Core simulator primitives, shared by every experiment in run_experiments.py:

  1. `Cache`            -- a generic N-way set-associative LRU cache that also
                           carries a per-line metadata slot (source: demand
                           or prefetch; used: bool; arrival_clock: int).
                           A fully-associative structure (the victim cache)
                           is just a `Cache` with `num_sets == 1`.
  2. `HierarchySim`      -- L1 -> (optional victim cache) -> L2 -> LLC -> DRAM,
                           strict inclusion-order lookup, cumulative
                           ("blocking") latency accounting, WITH a
                           prefetch-issue hook and enough per-line
                           provenance bookkeeping to compute prefetch
                           accuracy / coverage / timeliness / traffic and
                           useful-data-eviction (pollution) statistics.
  3. Three prefetcher classes: `NextLinePrefetcher`, `StridePrefetcher`,
                           `StreamPrefetcher`.

Modeling choices (stated explicitly, see report Section 2):
  - Latency accounting is a simple cumulative/serial model: a demand access
    that misses in L1 and hits in L2 costs L1_LAT + L2_LAT, etc. This is the
    same simplification used in the Exp-03 simulator. It is adequate for
    every experiment EXCEPT the blocking-vs-non-blocking / MSHR study, which
    is specifically ABOUT overlapping outstanding misses and therefore needs
    its own timing model (see mlp_model.py) -- a single cumulative-latency
    number cannot represent memory-level parallelism by construction.
  - Prefetched lines are inserted into the cache functionally at ISSUE time
    (so they correctly participate in LRU/eviction/pollution bookkeeping the
    moment they are fetched -- an early, unwanted prefetch pollutes the
    cache immediately, exactly as in a real design), but each prefetched
    line also carries a predicted `arrival_clock`. A later demand access to
    that line, if it occurs before `arrival_clock`, still has to wait out
    the remaining latency (a "late" prefetch: useful, but not timely). This
    gives a genuine timeliness signal without a full event-driven simulator.
  - Prefetches are issued for the L1 (aggressive: prefetch requests walk the
    full hierarchy and fill all the way up to L1), which is precisely what
    lets us observe L1 pollution/displacement effects, not just miss-count
    changes at L2/LLC.
"""

from __future__ import annotations

from collections import defaultdict

LINE_SIZE = 64
OFFSET_BITS = LINE_SIZE.bit_length() - 1  # 6


# ============================================================================
# 1. Generic N-way set-associative LRU cache with per-line metadata
# ============================================================================

class Cache:
    __slots__ = (
        "name", "ways", "num_sets", "set_mask", "index_bits", "latency",
        "tags", "lru", "meta", "clock", "hits", "misses", "capacity_bytes",
    )

```

```
    def __init__(self, name: str, size_bytes: int, line_size: int, ways: int, latency: int):
        self.name = name
        self.ways = ways
        self.capacity_bytes = size_bytes
        num_lines = size_bytes // line_size
        self.num_sets = max(1, num_lines // ways)
        assert self.num_sets & (self.num_sets - 1) == 0, (
            f"{name}: num_sets={self.num_sets} must be a power of two "
            f"(size={size_bytes}, ways={ways}, line={line_size})"
        )
        self.set_mask = self.num_sets - 1
        self.index_bits = self.num_sets.bit_length() - 1
        self.latency = latency

        self.tags = [[-1] * ways for _ in range(self.num_sets)]
        self.lru = [[0] * ways for _ in range(self.num_sets)]
        self.meta = [[None] * ways for _ in range(self.num_sets)]
        self.clock = 0
        self.hits = 0
        self.misses = 0

    def index_tag(self, line_id: int):
        return line_id & self.set_mask, line_id >> self.index_bits

    def probe(self, line_id: int):
        idx, tag = self.index_tag(line_id)
        row = self.tags[idx]
        for w in range(self.ways):
            if row[w] == tag:
                return True, idx, w
        return False, idx, None

    def touch(self, idx: int, way: int) -> None:
        self.clock += 1
        self.lru[idx][way] = self.clock

    def invalidate(self, idx: int, way: int) -> None:
        self.tags[idx][way] = -1
        self.meta[idx][way] = None

    def install(self, idx: int, tag: int, meta):
        """Insert (idx, tag) with metadata `meta`. Returns the evicted
        (old_line_id, old_meta) if a resident line had to be replaced, else
        None if an empty way was used."""
        self.clock += 1
        row_tags = self.tags[idx]
        row_lru = self.lru[idx]
        row_meta = self.meta[idx]
        for w in range(self.ways):
            if row_tags[w] == -1:
                row_tags[w] = tag
                row_lru[w] = self.clock
                row_meta[w] = meta
                return None
        victim = 0
        min_stamp = row_lru[0]
        for w in range(1, self.ways):
            if row_lru[w] < min_stamp:
                min_stamp = row_lru[w]
                victim = w
        old_tag = row_tags[victim]
        old_meta = row_meta[victim]
        old_line_id = (old_tag << self.index_bits) | idx
        row_tags[victim] = tag
        row_lru[victim] = self.clock
        row_meta[victim] = meta
        return (old_line_id, old_meta)

    def put(self, line_id: int, meta):
        idx, tag = self.index_tag(line_id)
        return self.install(idx, tag, meta)

    def occupancy(self) -> int:
```

```
        return sum(1 for row in self.tags for t in row if t != -1)


# ============================================================================
# 2. Prefetchers
# ============================================================================

class NextLinePrefetcher:
    """Fixed, stateless: on every triggering (miss) reference to line L,
    prefetch L+1. Distance is not configurable by design (that is the whole
    point of contrasting it with stride/stream)."""
    name = "next_line"

    def observe(self, stream_id, line_id: int, was_hit: bool):
        return (line_id + 1,)


class StridePrefetcher:
    """Per-stream (per access-stream identity, standing in for a per-PC
    stride table in real hardware) two-delta stride detector. Once the last
    two deltas agree, issues ONE prefetch `distance` strides ahead."""
    name = "stride"

    def __init__(self, distance: int, confirm: int = 1):
        self.distance = distance
        self.confirm = confirm
        self.state: dict = {}

    def observe(self, stream_id, line_id: int, was_hit: bool):
        st = self.state.get(stream_id)
        if st is None:
            self.state[stream_id] = {"last": line_id, "stride": None, "conf": 0}
            return ()
        cands = ()
        if st["last"] is not None:
            delta = line_id - st["last"]
            if delta != 0 and delta == st["stride"]:
                st["conf"] += 1
            else:
                st["conf"] = 0
            st["stride"] = delta
            if st["conf"] >= self.confirm and st["stride"]:
                cands = (line_id + self.distance * st["stride"],)
        st["last"] = line_id
        return cands


class StreamPrefetcher:
    """Per-stream run-ahead engine. Requires TWO consecutive matching deltas
    to confirm a stream (more conservative than the stride detector), then
    maintains a run-ahead window of `distance` lines: fills the whole window
    on confirmation, and thereafter advances it by one stride per confirmed
    step, mimicking a hardware stream buffer's steady-state behaviour."""
    name = "stream"

    def __init__(self, distance: int, confirm: int = 2):
        self.distance = distance
        self.confirm = confirm
        self.state: dict = {}

    def observe(self, stream_id, line_id: int, was_hit: bool):
        st = self.state.get(stream_id)
        if st is None:
            self.state[stream_id] = {"last": line_id, "stride": None, "conf": 0, "frontier": None}
            return ()
        cands = []
        delta = line_id - st["last"]
        if delta != 0 and delta == st["stride"]:
            st["conf"] += 1
        else:
            st["conf"] = 0
            st["frontier"] = None
        st["stride"] = delta
```

```
        if st["conf"] >= self.confirm and st["stride"]:
            if st["frontier"] is None:
                for k in range(1, self.distance + 1):
                    cands.append(line_id + k * st["stride"])
                st["frontier"] = line_id + self.distance * st["stride"]
            else:
                target = line_id + self.distance * st["stride"]
                if target != st["frontier"]:
                    cands.append(target)
                    st["frontier"] = target
        st["last"] = line_id
        return cands


# ============================================================================
# 3. Cache hierarchy simulator
# ============================================================================

class HierarchySim:
    def __init__(self, l1_cfg, l2_cfg, llc_cfg, dram_latency: int,
                 victim_entries: int = 0, victim_latency: int = 3,
                 prefetcher=None):
        self.l1 = Cache("L1", *l1_cfg)
        self.l2 = Cache("L2", *l2_cfg)
        self.llc = Cache("LLC", *llc_cfg)
        self.vc = (Cache("VC", victim_entries * LINE_SIZE, LINE_SIZE, victim_entries, victim_latency)
                   if victim_entries > 0 else None)
        self.dram_latency = dram_latency
        self.prefetcher = prefetcher

        self.clock = 0
        self.total_accesses = 0
        self.dram_accesses = 0
        self.dram_prefetch_accesses = 0

        self.prefetch_issued = 0
        self.prefetch_redundant = 0
        self.prefetch_useful = 0
        self.prefetch_timely = 0
        self.prefetch_late = 0
        self.prefetch_wasted_evicted = 0
        self.useful_data_evictions = 0

        # L1 hit/miss broken out per logical access-stream ("agg_state",
        # "ingest_records", ...) -- lets us see e.g. the aggregation
        # state's own residency/hit-rate collapse under pressure, which
        # the aggregate L1 miss rate alone would hide.
        self.stream_stats = defaultdict(lambda: {"hits": 0, "misses": 0})

    # ---- eviction bookkeeping ----------------------------------------
    def _account_eviction(self, evicted) -> None:
        if evicted is None:
            return
        _old_line_id, old_meta = evicted
        if old_meta is None:
            return
        if old_meta["source"] == "demand" and old_meta.get("used"):
            self.useful_data_evictions += 1
        elif old_meta["source"] == "prefetch" and not old_meta.get("used"):
            self.prefetch_wasted_evicted += 1

    def _fill_l1_from(self, line_id: int, meta: dict) -> None:
        evicted = self.l1.put(line_id, dict(meta))
        self._account_eviction(evicted)
        if evicted is not None and self.vc is not None:
            old_line_id, old_meta = evicted
            if old_meta is not None:
                ev2 = self.vc.put(old_line_id, old_meta)
                self._account_eviction(ev2)

    def _fill_l2_l1_from(self, line_id: int, meta: dict) -> None:
        ev2 = self.l2.put(line_id, dict(meta))
        self._account_eviction(ev2)
```

```
        self._fill_l1_from(line_id, meta)

    def _fill_llc_l2_l1_from(self, line_id: int, meta: dict) -> None:
        evL = self.llc.put(line_id, dict(meta))
        self._account_eviction(evL)
        self._fill_l2_l1_from(line_id, meta)

    # ---- demand access ------------------------------------------------
    def access(self, addr: int, stream_id="default") -> None:
        self.total_accesses += 1
        line_id = addr >> OFFSET_BITS
        now = self.clock

        hit1, idx1, way1 = self.l1.probe(line_id)
        ss = self.stream_stats[stream_id]
        ss["hits" if hit1 else "misses"] += 1
        if hit1:
            meta = self.l1.meta[idx1][way1]
            self.l1.touch(idx1, way1)
            self.l1.hits += 1
            cost = self.l1.latency
            if meta is not None:
                if meta["source"] == "prefetch" and not meta["used"]:
                    meta["used"] = True
                    self.prefetch_useful += 1
                    if now >= meta["arrival_clock"]:
                        self.prefetch_timely += 1
                    else:
                        self.prefetch_late += 1
                        cost += (meta["arrival_clock"] - now)
                else:
                    meta["used"] = True
            self.clock += cost
            was_hit = True
        else:
            self.l1.misses += 1
            was_hit = False
            served = False
            if self.vc is not None:
                hitv, idxv, wayv = self.vc.probe(line_id)
                if hitv:
                    self.vc.hits += 1
                    vmeta = self.vc.meta[idxv][wayv] or {"source": "demand", "used": True}
                    vmeta["used"] = True
                    self.vc.invalidate(idxv, wayv)
                    evicted = self.l1.put(line_id, vmeta)
                    self._account_eviction(evicted)
                    if evicted is not None:
                        old_line_id, old_meta = evicted
                        if old_meta is not None:
                            ev2 = self.vc.put(old_line_id, old_meta)
                            self._account_eviction(ev2)
                    self.clock += self.l1.latency + self.vc.latency
                    served = True
                else:
                    self.vc.misses += 1

            if not served:
                hit2, idx2, way2 = self.l2.probe(line_id)
                if hit2:
                    meta = self.l2.meta[idx2][way2] or {"source": "demand", "used": True}
                    meta["used"] = True
                    self.l2.touch(idx2, way2)
                    self.l2.hits += 1
                    self._fill_l1_from(line_id, meta)
                    self.clock += self.l1.latency + self.l2.latency
                else:
                    self.l2.misses += 1
                    hit3, idx3, way3 = self.llc.probe(line_id)
                    if hit3:
                        meta = self.llc.meta[idx3][way3] or {"source": "demand", "used": True}
                        meta["used"] = True
                        self.llc.touch(idx3, way3)
```

```
                        self.llc.hits += 1
                        self._fill_l2_l1_from(line_id, meta)
                        self.clock += self.l1.latency + self.l2.latency + self.llc.latency
                    else:
                        self.llc.misses += 1
                        self.dram_accesses += 1
                        self._fill_llc_l2_l1_from(line_id, {"source": "demand", "used": True})
                        self.clock += self.l1.latency + self.l2.latency + self.llc.latency + self.
    dram_latency

        if self.prefetcher is not None:
            for cand in self.prefetcher.observe(stream_id, line_id, was_hit):
                if cand >= 0:
                    self._issue_prefetch(cand)

    # ---- prefetch issue (async w.r.t. the demand clock, but walks the
    # same functional hierarchy so it generates real traffic/evictions) ---
    def _issue_prefetch(self, line_id: int) -> None:
        hit1, _, _ = self.l1.probe(line_id)
        if hit1:
            self.prefetch_redundant += 1
            return
        if self.vc is not None:
            hitv, _, _ = self.vc.probe(line_id)
            if hitv:
                self.prefetch_redundant += 1
                return

        self.prefetch_issued += 1
        now = self.clock
        hit2, _, _ = self.l2.probe(line_id)
        if hit2:
            lat = self.l1.latency + self.l2.latency
            meta = {"source": "prefetch", "used": False, "arrival_clock": now + lat}
            self._fill_l1_from(line_id, meta)
            return
        hit3, _, _ = self.llc.probe(line_id)
        if hit3:
            lat = self.l1.latency + self.l2.latency + self.llc.latency
            meta = {"source": "prefetch", "used": False, "arrival_clock": now + lat}
            self._fill_l2_l1_from(line_id, meta)
            return
        self.dram_prefetch_accesses += 1
        lat = self.l1.latency + self.l2.latency + self.llc.latency + self.dram_latency
        meta = {"source": "prefetch", "used": False, "arrival_clock": now + lat}
        self._fill_llc_l2_l1_from(line_id, meta)

    # ---- reporting ------------------------------------------------
    def snapshot(self) -> dict:
        def rate(h, m):
            return 100.0 * m / (h + m) if (h + m) else 0.0
        return {
            "total_accesses": self.total_accesses,
            "clock": self.clock,
            "amat": self.clock / self.total_accesses if self.total_accesses else 0.0,
            "l1_hits": self.l1.hits, "l1_misses": self.l1.misses,
            "l1_miss_pct": rate(self.l1.hits, self.l1.misses),
            "vc_hits": self.vc.hits if self.vc else 0,
            "vc_misses": self.vc.misses if self.vc else 0,
            "l2_hits": self.l2.hits, "l2_misses": self.l2.misses,
            "l2_miss_pct": rate(self.l2.hits, self.l2.misses),
            "llc_hits": self.llc.hits, "llc_misses": self.llc.misses,
            "llc_miss_pct": rate(self.llc.hits, self.llc.misses),
            "dram_accesses": self.dram_accesses,
            "dram_prefetch_accesses": self.dram_prefetch_accesses,
            "prefetch_issued": self.prefetch_issued,
            "prefetch_redundant": self.prefetch_redundant,
            "prefetch_useful": self.prefetch_useful,
            "prefetch_timely": self.prefetch_timely,
            "prefetch_late": self.prefetch_late,
            "prefetch_wasted_evicted": self.prefetch_wasted_evicted,
            "prefetch_accuracy_pct": 100.0 * self.prefetch_useful / self.prefetch_issued if self.
    prefetch_issued else 0.0,
```

```
            "prefetch_timeliness_pct": 100.0 * self.prefetch_timely / self.prefetch_useful if self.
    prefetch_useful else 0.0,
            "useful_data_evictions": self.useful_data_evictions,
        }


# ============================================================================
# 4. Standard processor-model parameters (Sapphire Rapids, simplified --
#    see report Section 1 for the exact caveats on these numbers)
# ============================================================================

L1_SIZE = 48 * 1024        # 48 KB
L1_WAYS = 12
L1_LATENCY = 5

L2_SIZE = 2 * 1024 * 1024  # 2 MB
L2_WAYS = 16
L2_LATENCY = 16

LLC_SIZE = 15 * 1024 * 1024  # 15 MiB aggregate (8 x 1.875 MB)
LLC_WAYS = 15
LLC_LATENCY = 60

DRAM_LATENCY = 200

VC_LATENCY = 3


def baseline_kwargs(**overrides):
    cfg = dict(
        l1_cfg=(L1_SIZE, LINE_SIZE, L1_WAYS, L1_LATENCY),
        l2_cfg=(L2_SIZE, LINE_SIZE, L2_WAYS, L2_LATENCY),
        llc_cfg=(LLC_SIZE, LINE_SIZE, LLC_WAYS, LLC_LATENCY),
        dram_latency=DRAM_LATENCY,
        victim_entries=0,
        victim_latency=VC_LATENCY,
        prefetcher=None,
    )
    cfg.update(overrides)
    return cfg
```

Listing 2: `src/workload.py` – the five-stage pipeline address-stream generators

```
"""
Exp-04 workload: a five-stage synthetic data-processing pipeline.

  1. Data Ingestion     -- sequential scan of a record array (much larger
                           than any private cache) interleaved with repeated
                           touches of a small, reused metadata/parameter
                           block.
  2. Feature Extraction -- per-record field extraction at a CONFIGURABLE
                           record-level stride (contiguous -> large),
                           forward or backward, touching a fixed number of
                           records regardless of stride so "useful work"
                           stays constant while only the ADDRESS PATTERN
                           changes.
  3. Feature Aggregation-- small, frequently-reused accumulator state,
                           updated every `agg_batch` input records (batch
                           size controls reuse distance / interleave
                           pressure).
  4. Lookup & Scoring   -- an index structure accessed with a configurable
                           mix of pseudo-random indices and a periodic,
                           partially-predictable regular component.
  5. Result Generation  -- two variants of equal "useful work":
                              (a) a dependent pointer-chase (each address
                                  depends on the previous "load"), and
                              (b) K independent streams, round-robin
                                  interleaved so consecutive accesses are
                                  mutually independent.

All generators yield `(address, stream_id)` pairs. `stream_id` stands in for
"the load instruction/PC that issued this access" -- the granularity real
stride/stream prefetchers key their per-stream state on. Every stage is a
```

```
plain Python generator (no data values are modeled, only the address
stream -- the same convention used in Assignment 1 / Assignment 3), except
the dependent-chain result-generation stream, whose *visitation order* is a
precomputed random cyclic permutation: this is the standard way pointer-
chasing benchmarks (e.g. lat_mem_rd) are modeled without simulating actual
memory contents, and it preserves the property that matters for this
assignment -- each hop is not predictable from the previous address, and by
construction the next request cannot be issued before the previous one
"returns".
"""

from __future__ import annotations

import random
from dataclasses import dataclass, field


# ============================================================================
# Layout helpers -- disjoint byte-address regions, one per array
# ============================================================================

@dataclass
class Layout:
    bases: dict = field(default_factory=dict)
    _cursor: int = 0

    def alloc(self, name: str, nbytes: int) -> int:
        base = self._cursor
        self.bases[name] = base
        self._cursor += nbytes
        return base

    def __getitem__(self, name):
        return self.bases[name]


# ============================================================================
# Stage 1: Data Ingestion
# ============================================================================

RECORD_BYTES = 128
FIELD_BYTES = 8
FIELDS_PER_RECORD = RECORD_BYTES // FIELD_BYTES  # 16
METADATA_BYTES = 4096  # small, reused every record


def stage_ingestion(layout: Layout, n_records: int, metadata_touches: int = 2):
    """Sequential record scan; the metadata/params block is re-touched
    `metadata_touches` times per record (stands in for configurable
    per-record computation intensity that repeatedly consults shared
    parameters)."""
    rec_base = layout["records"]
    meta_base = layout["metadata"]
    n_meta_lines = max(1, METADATA_BYTES // 64)
    for i in range(n_records):
        rb = rec_base + i * RECORD_BYTES
        for f in range(0, FIELDS_PER_RECORD, 8):  # touch every 8th field -> 2 lines/record
            yield rb + f * FIELD_BYTES, "ingest_records"
        for t in range(metadata_touches):
            yield meta_base + (t % n_meta_lines) * 64, "ingest_metadata"


# ============================================================================
# Stage 2: Feature Extraction
# ============================================================================

def stage_extraction(layout: Layout, n_records: int, extract_count: int,
                     stride_records: int, fields_per_record: int = 3,
                     forward: bool = True):
    """Touch `extract_count` records (regardless of stride, so useful work
    is constant), selecting record i_k = (k * stride_records) % n_records,
    k ascending (forward) or descending (backward). At each selected
    record, read `fields_per_record` fields."""
```

```
    rec_base = layout["records"]
    field_offsets = [f * (FIELD_BYTES * 3) for f in range(fields_per_record)]  # spread within record
    ks = range(extract_count) if forward else range(extract_count - 1, -1, -1)
    for k in ks:
        i = (k * stride_records) % n_records
        rb = rec_base + i * RECORD_BYTES
        for off in field_offsets:
            yield rb + (off % RECORD_BYTES), "extract"


# ============================================================================
# Stage 3: Feature Aggregation
# ============================================================================

def stage_aggregation(layout: Layout, n_records: int, agg_entries: int,
                      agg_batch: int, rng: random.Random):
    """For every `agg_batch` input records streamed past, touch one
    aggregation-state entry (small, hot, reused array)."""
    rec_base = layout["records"]
    agg_base = layout["agg_state"]
    entry_bytes = 8
    n_entries = agg_entries
    for i in range(n_records):
        rb = rec_base + i * RECORD_BYTES
        yield rb, "agg_input"
        if i % agg_batch == 0:
            slot = rng.randrange(n_entries)
            yield agg_base + slot * entry_bytes, "agg_state"


# ============================================================================
# Stage 4: Lookup & Scoring
# ============================================================================

def stage_lookup(layout: Layout, lookup_count: int, index_entries: int,
                 p_random: float, regular_period: int, rng: random.Random):
    """Mix of pseudo-random index accesses and a periodic, partially
    predictable regular component (recurring regularities)."""
    idx_base = layout["index"]
    entry_bytes = 32
    regular_cursor = 0
    for _ in range(lookup_count):
        if rng.random() < p_random:
            slot = rng.randrange(index_entries)
        else:
            slot = regular_cursor % index_entries
            regular_cursor += regular_period
        yield idx_base + slot * entry_bytes, "lookup"


# ============================================================================
# Stage 5: Result Generation -- dependent chain vs independent streams
# ============================================================================

def build_chain_permutation(chain_entries: int, rng: random.Random):
    """A random cyclic permutation over `chain_entries` slots: visiting it
    in order models pointer-chasing (each 'next' address is unpredictable
    from the current one, and cannot be known before the current load
    completes)."""
    order = list(range(chain_entries))
    rng.shuffle(order)
    return order


def stage_result_dependent(layout: Layout, chain_entries: int, steps: int,
                           permutation):
    base = layout["chain"]
    entry_bytes = 64  # one line per node
    for s in range(steps):
        slot = permutation[s % chain_entries]
        yield base + slot * entry_bytes, "resgen_dep"


```

```
def stage_result_independent(layout: Layout, k_streams: int, steps_per_stream: int,
                             chain_entries_per_stream: int, rng: random.Random):
    """K mutually-independent pointer-chase chains, round-robin interleaved
    one step per stream per round. Accesses WITHIN one stream are still a
    dependent chain (each is a genuine "next useful step" of that stream's
    own computation) -- what makes the streams "independent" is that stream
    k's next access never waits on stream j's outstanding request, so up to
    K requests (one per stream) can be truly outstanding at once. This is
    what makes memory-level parallelism here bounded by K, not unbounded --
    exactly what the blocking-vs-non-blocking / MSHR experiment needs to
    show a genuine plateau."""
    base = layout["streams"]
    entry_bytes = 64
    stream_span = chain_entries_per_stream * entry_bytes
    perms = []
    for _ in range(k_streams):
        p = list(range(chain_entries_per_stream))
        rng.shuffle(p)
        perms.append(p)
    for r in range(steps_per_stream):
        for k in range(k_streams):
            slot = perms[k][r % chain_entries_per_stream]
            yield base + k * stream_span + slot * entry_bytes, f"resgen_indep_{k}"


# ============================================================================
# Config + full-pipeline / combined-pressure drivers
# ============================================================================

@dataclass
class WorkloadConfig:
    n_records: int = 120_000
    metadata_touches: int = 2

    extract_count: int = 90_000
    extract_stride_records: int = 1
    extract_fields: int = 3
    extract_forward: bool = True

    agg_entries: int = 512
    agg_batch: int = 4

    index_entries: int = 300_000
    lookup_count: int = 90_000
    lookup_p_random: float = 0.6
    lookup_regular_period: int = 7

    chain_entries: int = 60_000
    resgen_steps: int = 60_000
    k_streams: int = 8

    seed: int = 42


def make_layout(cfg: WorkloadConfig) -> Layout:
    lay = Layout()
    lay.alloc("records", cfg.n_records * RECORD_BYTES)
    lay.alloc("metadata", METADATA_BYTES)
    lay.alloc("agg_state", cfg.agg_entries * 8)
    lay.alloc("index", cfg.index_entries * 32)
    lay.alloc("chain", cfg.chain_entries * 64)
    lay.alloc("streams", cfg.k_streams * cfg.resgen_steps * 64 * 2 + 4096)
    return lay


def full_pipeline_stream(cfg: WorkloadConfig, resgen_variant: str = "independent"):
    """Yields (addr, stream_id, stage) for the complete five-stage pipeline,
    stage by stage, in order -- WITHOUT clearing cache state between
    stages (so later stages inherit whatever residency earlier stages left
    behind, as specified)."""
    rng = random.Random(cfg.seed)
    lay = make_layout(cfg)

```

```
    for addr, sid in stage_ingestion(lay, cfg.n_records, cfg.metadata_touches):
        yield addr, sid, "1_ingestion"
    for addr, sid in stage_extraction(lay, cfg.n_records, cfg.extract_count,
                                      cfg.extract_stride_records, cfg.extract_fields,
                                      cfg.extract_forward):
        yield addr, sid, "2_extraction"
    for addr, sid in stage_aggregation(lay, cfg.n_records, cfg.agg_entries,
                                       cfg.agg_batch, rng):
        yield addr, sid, "3_aggregation"
    for addr, sid in stage_lookup(lay, cfg.lookup_count, cfg.index_entries,
                                  cfg.lookup_p_random, cfg.lookup_regular_period, rng):
        yield addr, sid, "4_lookup"
    if resgen_variant == "independent":
        steps_per_stream = cfg.resgen_steps // cfg.k_streams
        chain_per_stream = max(2, cfg.chain_entries // cfg.k_streams)
        for addr, sid in stage_result_independent(lay, cfg.k_streams, steps_per_stream,
                                                  chain_per_stream, rng):
            yield addr, sid, "5_result_gen"
    else:
        perm = build_chain_permutation(cfg.chain_entries, rng)
        for addr, sid in stage_result_dependent(lay, cfg.chain_entries, cfg.resgen_steps, perm):
            yield addr, sid, "5_result_gen"


def combined_pressure_stream(cfg: WorkloadConfig, pressure: int, max_pressure: int = 64):
    """Ingestion + Extraction + Aggregation, ROUND-ROBIN interleaved at
    record granularity (rather than run sequentially), so continuously
    arriving input data genuinely competes, cycle by cycle, with the
    frequently-reused aggregation state for cache residency. ‘pressure‘
    controls how many FRESH, never-before-touched foreign lines (drawn
    from a dedicated, monotonically-advancing filler region -- distinct
    from any bounded, wrap-around array) are injected between successive
    aggregation-state touches: this directly and monotonically controls
    the reuse DISTANCE separating one visit to the hot state from the
    next, with no risk of an accidental short address-cycle (e.g. a
    stride sharing a factor with a bounded array’s length) creating
    spurious extra reuse at some pressure levels and not others."""
    rng = random.Random(cfg.seed)
    lay = make_layout(cfg)
    rec_base = lay["records"]
    agg_base = lay["agg_state"]
    filler_base = lay.alloc("pressure_filler", max_pressure * cfg.n_records * 64 + 4096)
    n_records = cfg.n_records
    agg_batch = max(1, cfg.agg_batch)
    cursor = 0

    for i in range(n_records):
        rb = rec_base + i * RECORD_BYTES
        yield rb, "ingest_records"
        yield rb + 64, "ingest_records"
        for _ in range(pressure):
            yield filler_base + cursor * 64, "extract"
            cursor += 1
        if i % agg_batch == 0:
            slot = rng.randrange(cfg.agg_entries)
            yield agg_base + slot * 8, "agg_state"
```

Listing 3: `src/mlp_model.py` – MSHR / non-blocking-cache timing model

```
"""
Blocking vs. non-blocking cache / MSHR timing model, used ONLY for the
Result-Generation experiment (Section 7 of the report).

Why a separate model from ‘HierarchySim‘’s own cumulative clock: that clock
is deliberately a *blocking*, single-outstanding-miss model (every access,
hit or miss, is fully serialized). Memory-level parallelism is precisely the
ability of MULTIPLE misses to be outstanding and completing concurrently,
which a single running cycle counter cannot represent. So this module takes
the hit/miss CLASSIFICATION (and the latency each access would cost in
isolation) from ‘HierarchySim‘, and re-times the access sequence under an
explicit MSHR model:

  - ‘mshr_count‘ outstanding-miss slots, each tracked by its "free at" cycle.
```

```
  - A demand HIT never needs an MSHR; it is charged its L1 latency and the
    core's issue point advances by that amount (simple in-order issue).
  - A demand MISS needs a free MSHR slot. If one is free, the request starts
    immediately (possibly overlapping previously issued, still-outstanding
    misses); if all `mshr_count` slots are busy, issuing STALLS until one
    frees (the "no free MSHR -> stall" structural hazard).
  - `dependent=True` additionally forces the core to wait for the miss's
    OWN completion before issuing anything else (the next address is not
    known until this load returns) -- this is true regardless of
    `mshr_count`, which is exactly the point: extra MSHRs cannot help a
    request stream that has no independence to exploit.
  - `dependent=False` lets the core keep issuing subsequent (independent)
    misses into other free MSHR slots while earlier ones are still
    outstanding -- multiple misses overlap in time, hiding their latency
    behind one another (memory-level parallelism).

`mshr_count=1` with `dependent=False` degenerates to a fully blocking
cache: only one miss can ever be outstanding, so independent streams gain
nothing over the dependent case -- exactly the textbook definition of a
blocking cache.
"""

from __future__ import annotations

import heapq

from cache_core import HierarchySim


def classify_stream(addr_stream, hierarchy_kwargs):
    """Run the address stream through a baseline (no VC, no prefetch)
    HierarchySim purely to classify each access as a hit or a miss and to
    record the latency it would cost IN ISOLATION (i.e. the per-access
    delta of the hierarchy's own blocking clock) -- this delta already
    encodes which level served the request (L1/L2/LLC/DRAM) via the
    cumulative-latency convention used everywhere else in this project."""
    sim = HierarchySim(**hierarchy_kwargs)
    lats = []
    sids = []
    for addr, sid in addr_stream:
        before = sim.clock
        sim.access(addr, sid)
        lats.append(sim.clock - before)
        sids.append(sid)
    return lats, sids, sim.snapshot(), sim.l1.latency


def simulate_mshr(lats, stream_ids, mshr_count: int, l1_latency: int):
    """General timing model with two overlapping constraints:

      1. WITHIN one stream_id, accesses are dependent -- the next access on
         that stream cannot issue before the previous one on that SAME
         stream has completed (`stream_ready[sid]`).
      2. ACROSS all streams, at most `mshr_count` misses may be outstanding
         at once (the shared MSHR resource, `heap`); a miss that finds no
         free slot stalls until one frees.
      3. A single global issue port (>= 1 cycle between any two issues) is
         also modeled, but with realistic MSHR counts this is never the
         binding constraint here -- (1) and (2) are.

    Pass every access with the SAME stream_id to get the fully-serialized
    "dependent chain" behaviour (case 1 alone forces total serialization,
    irrespective of `mshr_count`). Pass K distinct stream_ids (round-robin
    interleaved) to get the "K independent streams" behaviour, where
    increasing `mshr_count` helps only up to about K before saturating,
    since at most K requests (one per stream) can ever be simultaneously
    ready to issue.
    """
    heap = [0] * max(1, mshr_count)
    heapq.heapify(heap)
    stream_ready: dict = {}
    global_issue = 0
    max_completion = 0
```

```
    n_misses = 0
    n_hits = 0
    miss_service_sum = 0

    for lat, sid in zip(lats, stream_ids):
        ready = stream_ready.get(sid, 0)
        if global_issue > ready:
            ready = global_issue
        is_miss = lat > l1_latency
        if not is_miss:
            n_hits += 1
            finish = ready + lat
            stream_ready[sid] = finish
            global_issue = ready + 1
            if finish > max_completion:
                max_completion = finish
            continue

        n_misses += 1
        miss_service_sum += lat
        earliest_free = heap[0]
        start = ready if ready > earliest_free else earliest_free
        completion = start + lat
        heapq.heapreplace(heap, completion)
        stream_ready[sid] = completion
        global_issue = start + 1
        if completion > max_completion:
            max_completion = completion

    total_cycles = max(global_issue, max_completion)
    avg_miss_service = miss_service_sum / n_misses if n_misses else 0.0
    return {
        "total_cycles": total_cycles,
        "n_accesses": len(lats),
        "n_hits": n_hits,
        "n_misses": n_misses,
        "avg_miss_service_cycles": avg_miss_service,
        "cycles_per_access": total_cycles / len(lats) if lats else 0.0,
    }
```

Listing 4: `run_experiments.py` – experiment driver, output writers

```
"""
Exp-04: When Cache Optimizations Make the Processor Slower -- experiment
driver. Runs the seven experiment groups (A-G) described in the report,
writes CSV/JSON results, matplotlib PNG plots, an append-only master
ledger, and a LaTeX tables fragment (‘results/tables.tex‘) that the report
\\input{}s directly.

Run with:  python3 run_experiments.py
"""

from __future__ import annotations

import csv
import json
import random
import sys
import time
from pathlib import Path

sys.path.insert(0, str(Path(__file__).parent / "src"))

import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

import cache_core
import workload
import mlp_model

RESULTS_DIR = Path(__file__).parent / "results"
PLOTS_DIR = RESULTS_DIR / "plots"
```

```
SEED = 42


# ============================================================================
# 0. Workload configurations
# ============================================================================

DEFAULT_CFG = workload.WorkloadConfig(
    n_records=80_000, metadata_touches=2,
    extract_count=60_000, extract_stride_records=1, extract_fields=3, extract_forward=True,
    agg_entries=512, agg_batch=4,
    index_entries=200_000, lookup_count=60_000, lookup_p_random=0.6, lookup_regular_period=7,
    chain_entries=40_000, resgen_steps=40_000, k_streams=8,
    seed=SEED,
)

PRESSURE_CFG = workload.WorkloadConfig(
    n_records=40_000, metadata_touches=2,
    extract_count=1, extract_stride_records=1, extract_fields=1, extract_forward=True,
    agg_entries=256, agg_batch=4,
    index_entries=1_000, lookup_count=1, lookup_p_random=0.6, lookup_regular_period=7,
    chain_entries=1_000, resgen_steps=800, k_streams=8,
    seed=SEED,
)


# ============================================================================
# 1. Generic helpers
# ============================================================================

_COUNTER_KEYS = (
    "total_accesses", "l1_hits", "l1_misses", "vc_hits", "vc_misses",
    "l2_hits", "l2_misses", "llc_hits", "llc_misses", "dram_accesses",
    "dram_prefetch_accesses", "prefetch_issued", "prefetch_redundant",
    "prefetch_useful", "prefetch_timely", "prefetch_late",
    "prefetch_wasted_evicted", "useful_data_evictions", "clock",
)


def delta_snapshot(before: dict, after: dict) -> dict:
    d = {k: after[k] - before[k] for k in _COUNTER_KEYS}
    acc = d["total_accesses"]
    d["amat"] = d["clock"] / acc if acc else 0.0
    l1h, l1m = d["l1_hits"], d["l1_misses"]
    d["l1_miss_pct"] = 100.0 * l1m / (l1h + l1m) if (l1h + l1m) else 0.0
    l2h, l2m = d["l2_hits"], d["l2_misses"]
    d["l2_miss_pct"] = 100.0 * l2m / (l2h + l2m) if (l2h + l2m) else 0.0
    lh, lm = d["llc_hits"], d["llc_misses"]
    d["llc_miss_pct"] = 100.0 * lm / (lh + lm) if (lh + lm) else 0.0
    pi, pu, pt = d["prefetch_issued"], d["prefetch_useful"], d["prefetch_timely"]
    d["prefetch_accuracy_pct"] = 100.0 * pu / pi if pi else 0.0
    d["prefetch_timeliness_pct"] = 100.0 * pt / pu if pu else 0.0
    return d


def expected_stage_counts(cfg: workload.WorkloadConfig, resgen_variant: str) -> dict:
    ingest = cfg.n_records * (2 + cfg.metadata_touches)
    extract = cfg.extract_count * cfg.extract_fields
    agg = cfg.n_records + (cfg.n_records + cfg.agg_batch - 1) // cfg.agg_batch
    lookup = cfg.lookup_count
    if resgen_variant == "independent":
        steps_per_stream = cfg.resgen_steps // cfg.k_streams
        resgen = steps_per_stream * cfg.k_streams
    else:
        resgen = cfg.resgen_steps
    return {"1_ingestion": ingest, "2_extraction": extract, "3_aggregation": agg,
            "4_lookup": lookup, "5_result_gen": resgen}


def run_pipeline_staged(cfg: workload.WorkloadConfig, resgen_variant: str = "independent",
                        cold_frac: float = 0.2, **hkwargs_overrides):
    """Runs the complete 5-stage pipeline through one HierarchySim (state
    carried across stages, never reset), returning per-stage stats AND a
```

```
    cold(first `cold_frac`)-vs-steady(rest) split within each stage."""
    hkwargs = cache_core.baseline_kwargs(**hkwargs_overrides)
    sim = cache_core.HierarchySim(**hkwargs)
    expected = expected_stage_counts(cfg, resgen_variant)

    stage_rows, cold_rows = [], []
    last_stage = None
    stage_before = None
    stage_count = 0
    cold_target = None
    cold_snap = None

    def close_stage():
        after = sim.snapshot()
        row = {"stage": last_stage, "n_accesses_expected": expected[last_stage]}
        row.update(delta_snapshot(stage_before, after))
        stage_rows.append(row)
        if cold_snap is not None:
            r1 = {"stage": last_stage + "_cold_0-20pct"}
            r1.update(delta_snapshot(stage_before, cold_snap))
            r2 = {"stage": last_stage + "_steady_20-100pct"}
            r2.update(delta_snapshot(cold_snap, after))
            cold_rows.append(r1)
            cold_rows.append(r2)

    for addr, sid, stage in workload.full_pipeline_stream(cfg, resgen_variant):
        if stage != last_stage:
            if last_stage is not None:
                close_stage()
            last_stage = stage
            stage_before = sim.snapshot()
            stage_count = 0
            cold_target = max(1, round(cold_frac * expected[stage]))
            cold_snap = None
        sim.access(addr, sid)
        stage_count += 1
        if cold_snap is None and stage_count >= cold_target:
            cold_snap = sim.snapshot()
    close_stage()

    overall = sim.snapshot()
    return {"stages": stage_rows, "cold_steady": cold_rows, "overall": overall, "sim": sim}


def pct(n, d):
    return 100.0 * n / d if d else 0.0


# ============================================================================
# 2. Experiment A -- baseline characterization
# ============================================================================

def exp_a_baseline():
    print("\n=== Exp A: baseline characterization ===")
    res = run_pipeline_staged(DEFAULT_CFG, resgen_variant="independent")
    for row in res["stages"]:
        print(f"    {row['stage']:<14s} acc={row['total_accesses']:>8,d}  "
              f"L1miss%={row['l1_miss_pct']:5.2f}  L2miss%={row['l2_miss_pct']:5.2f}  "
              f"LLCmiss%={row['llc_miss_pct']:5.2f}  AMAT={row['amat']:6.2f}")
    print(f"    OVERALL clock={res['overall']['clock']:,}  AMAT={res['overall']['amat']:.2f}  "
          f"L1miss%={res['overall']['l1_miss_pct']:.2f}")
    return res


# ============================================================================
# 3. Experiment B -- victim cache
# ============================================================================

VC_SIZES = (0, 8, 16, 32, 64)


def exp_b_victim_cache():
    print("\n=== Exp B: victim cache sweep ===")
```

```
    rows = []
    for vc in VC_SIZES:
        res = run_pipeline_staged(DEFAULT_CFG, resgen_variant="independent", victim_entries=vc)
        s = dict(res["overall"])
        s["victim_entries"] = vc
        s["extra_storage_bytes"] = vc * 64
        rows.append(s)
        print(f"    VC={vc:>3d} entries  clock={s['clock']:>10,d}  L1miss%={s['l1_miss_pct']:5.2f}  "
              f"vc_hits={s['vc_hits']:>7,d}  AMAT={s['amat']:.2f}")

    # comparable-storage alternative: L1 stays 12-way, but add 64 extra
    # LINES worth of capacity via one extra way (13-way) -- same +4096 B
    # as the 64-entry victim cache, so the two are a fair, equal-budget
    # comparison.
    l1_bigger = (cache_core.L1_SIZE + 64 * 64, cache_core.LINE_SIZE, 13, cache_core.L1_LATENCY)
    res_assoc = run_pipeline_staged(DEFAULT_CFG, resgen_variant="independent", l1_cfg=l1_bigger)
    assoc_row = dict(res_assoc["overall"])
    assoc_row["label"] = "L1_13way_(+4KB, no VC)"
    print(f"    L1 13-way (+4KB, no VC): clock={assoc_row['clock']:,}  L1miss%={assoc_row['l1_miss_pct
    ']:.2f}")

    return {"vc_sweep": rows, "assoc_compare": assoc_row}


def exp_b_conflict_microbench():
    """The full pipeline's misses are overwhelmingly cold/capacity misses on
    a streaming dataset far larger than any cache -- a poor showcase for a
    victim cache, which specifically targets CONFLICT misses (repeated
    thrashing among a *few* addresses that alias to the same set of a
    limited-associativity cache). This isolates that behaviour directly:
    N = L1_ways + 3 addresses, all mapping to the SAME L1 set, accessed in
    a round-robin loop -- a classic pathological pattern that guarantees
    every access misses once N exceeds the set's associativity, and that a
    victim cache of >= (N - ways) entries should almost fully absorb."""
    print("\n=== Exp B (supplementary): pathological same-set conflict microbenchmark ===")
    ways = cache_core.L1_WAYS
    num_sets = 64  # L1's num_sets at (48KB, 12-way, 64B lines)
    extra = 3
    N = ways + extra
    ROUNDS = 3000
    addrs = [(t * num_sets) * cache_core.LINE_SIZE for t in range(N)]
    rows = []
    for vc in (0, 1, 2, 3, 4, 8):
        sim = cache_core.HierarchySim(**cache_core.baseline_kwargs(victim_entries=vc))
        for _ in range(ROUNDS):
            for a in addrs:
                sim.access(a, "conflict_set0")
        snap = sim.snapshot()
        row = {"victim_entries": vc, "N_lines": N, "l1_ways": ways,
               "l1_miss_pct": snap["l1_miss_pct"], "clock": snap["clock"], "vc_hits": snap["vc_hits"]}
        rows.append(row)
        print(f"    VC={vc:>2d}  N={N} lines vs {ways}-way set  L1miss%={row['l1_miss_pct']:6.2f}  "
              f"clock={row['clock']:,}  vc_hits={row['vc_hits']:,}")
    return rows


# ============================================================================
# 4. Experiment C -- prefetcher comparison
# ============================================================================

PREFETCH_FACTORIES = (
    ("none", lambda: None),
    ("next_line", lambda: cache_core.NextLinePrefetcher()),
    ("stride_d1", lambda: cache_core.StridePrefetcher(distance=1)),
    ("stride_d4", lambda: cache_core.StridePrefetcher(distance=4)),
    ("stride_d8", lambda: cache_core.StridePrefetcher(distance=8)),
    ("stream_d4", lambda: cache_core.StreamPrefetcher(distance=4)),
    ("stream_d8", lambda: cache_core.StreamPrefetcher(distance=8)),
    ("stream_d16", lambda: cache_core.StreamPrefetcher(distance=16)),
)


def exp_c_prefetchers():
```

```
    print("\n=== Exp C: prefetcher comparison (full pipeline) ===")
    rows = []
    baseline_misses = None
    for label, factory in PREFETCH_FACTORIES:
        res = run_pipeline_staged(DEFAULT_CFG, resgen_variant="independent", prefetcher=factory())
        s = dict(res["overall"])
        s["prefetcher"] = label
        if label == "none":
            baseline_misses = s["l1_misses"]
        s["baseline_l1_misses"] = baseline_misses
        s["coverage_pct"] = pct(s["prefetch_useful"], baseline_misses)
        rows.append(s)
        print(f"    {label:<12s} clock={s['clock']:>10,d}  L1miss%={s['l1_miss_pct']:5.2f}  "
              f"acc%={s['prefetch_accuracy_pct']:5.1f}  cov%={s['coverage_pct']:5.1f}  "
              f"timely%={s['prefetch_timeliness_pct']:5.1f}  issued={s['prefetch_issued']:>7,d}")
    return rows


# ============================================================================
# 5. Experiment D -- prefetch distance sweep & unintended slowdown
# ============================================================================

DISTANCES = (1, 2, 4, 8, 16, 32, 64)


def exp_d_distance_sweep():
    print("\n=== Exp D: prefetch distance sweep (full pipeline) ===")
    rows = []
    for kind in ("stride", "stream"):
        for dist in DISTANCES:
            pf = (cache_core.StridePrefetcher(distance=dist) if kind == "stride"
                  else cache_core.StreamPrefetcher(distance=dist))
            res = run_pipeline_staged(DEFAULT_CFG, resgen_variant="independent", prefetcher=pf)
            s = dict(res["overall"])
            s["kind"] = kind
            s["distance"] = dist
            rows.append(s)
            print(f"    {kind:<7s} dist={dist:>3d}  clock={s['clock']:>10,d}  "
                  f"L1miss%={s['l1_miss_pct']:5.2f}  wasted_evicted={s['prefetch_wasted_evicted']:>6,d}
      "
                  f"useful_evictions={s['useful_data_evictions']:>7,d}")

    # no-prefetch reference point (distance = 0)
    res0 = run_pipeline_staged(DEFAULT_CFG, resgen_variant="independent", prefetcher=None)
    s0 = dict(res0["overall"])
    s0["kind"] = "none"
    s0["distance"] = 0
    rows.insert(0, s0)
    return rows


# ============================================================================
# 6. Experiment E -- pollution & thrashing (ingestion+extraction+aggregation,
#    genuinely interleaved so continuously arriving data competes with the
#    small reused aggregation state for cache residency)
# ============================================================================

PRESSURE_LEVELS = (1, 2, 4, 8, 16, 32)


def _run_pressure(pressure: int, victim_entries: int = 0, prefetcher=None):
    hkwargs = cache_core.baseline_kwargs(victim_entries=victim_entries, prefetcher=prefetcher)
    sim = cache_core.HierarchySim(**hkwargs)
    for addr, sid in workload.combined_pressure_stream(PRESSURE_CFG, pressure):
        sim.access(addr, sid)
    snap = sim.snapshot()
    agg = sim.stream_stats.get("agg_state", {"hits": 0, "misses": 0})
    agg_total = agg["hits"] + agg["misses"]
    agg_hit_pct = pct(agg["hits"], agg_total)
    return snap, agg_hit_pct, agg_total


def exp_e_pollution_thrashing():
```

```
    print("\n=== Exp E: cache pollution & thrashing (interleaved pipeline) ===")
    base_rows = []
    for p in PRESSURE_LEVELS:
        snap, agg_hit_pct, agg_total = _run_pressure(p)
        row = dict(snap)
        row["pressure"] = p
        row["agg_state_hit_pct"] = agg_hit_pct
        row["agg_state_accesses"] = agg_total
        row["config"] = "baseline"
        base_rows.append(row)
        print(f"    pressure={p:>3d}  agg_state_hit%={agg_hit_pct:6.2f}  overall_L1miss%={row['
    l1_miss_pct']:5.2f}")

    mitigated_rows = []
    hi_pressure = PRESSURE_LEVELS[-2:]  # the two highest-pressure points
    for p in hi_pressure:
        for label, kw in (
            ("+victim32", {"victim_entries": 32}),
            ("+stream_pf_d4", {"prefetcher": cache_core.StreamPrefetcher(distance=4)}),
            ("+victim32+stream_pf_d4", {"victim_entries": 32, "prefetcher": cache_core.StreamPrefetcher
    (distance=4)}),
        ):
            snap, agg_hit_pct, agg_total = _run_pressure(p, **kw)
            row = dict(snap)
            row["pressure"] = p
            row["agg_state_hit_pct"] = agg_hit_pct
            row["agg_state_accesses"] = agg_total
            row["config"] = label
            mitigated_rows.append(row)
            print(f"    pressure={p:>3d} {label:<24s} agg_state_hit%={agg_hit_pct:6.2f}  "
                  f"overall_L1miss%={row['l1_miss_pct']:5.2f}")

    return {"baseline": base_rows, "mitigation": mitigated_rows}


# ============================================================================
# 7. Experiment F -- blocking vs. non-blocking cache / MSHRs (Result Gen.)
# ============================================================================

MSHR_COUNTS = (1, 2, 4, 8, 16, 32, 64)
F_STEPS = 24_000
F_K = 8
F_CHAIN = 8_000


def exp_f_mshr():
    print("\n=== Exp F: blocking vs. non-blocking / MSHR sweep (Result Generation) ===")
    rng = random.Random(SEED)
    lay = workload.Layout()
    lay.alloc("records", 1)
    lay.alloc("metadata", 1)
    lay.alloc("agg_state", 1)
    lay.alloc("index", 1)
    lay.alloc("chain", F_CHAIN * 64)
    lay.alloc("streams", F_K * (F_STEPS // F_K) * 64 * 4 + 4096)

    perm = workload.build_chain_permutation(F_CHAIN, rng)
    dep_stream = list(workload.stage_result_dependent(lay, F_CHAIN, F_STEPS, perm))
    indep_stream = list(workload.stage_result_independent(
        lay, F_K, F_STEPS // F_K, max(2, F_CHAIN // F_K), rng))

    dep_lats, dep_sids, dep_snap, l1_lat = mlp_model.classify_stream(dep_stream, cache_core.
    baseline_kwargs())
    indep_lats, indep_sids, indep_snap, _ = mlp_model.classify_stream(indep_stream, cache_core.
    baseline_kwargs())

    print(f"    dependent chain:   {len(dep_lats):,} accesses, "
          f"L1 miss%={pct(dep_snap['l1_misses'], dep_snap['l1_hits'] + dep_snap['l1_misses']):.1f}")
    print(f"    independent streams: {len(indep_lats):,} accesses, "
          f"L1 miss%={pct(indep_snap['l1_misses'], indep_snap['l1_hits'] + indep_snap['l1_misses']):.1f
    }")

    rows = []
```

```
    for m in MSHR_COUNTS:
        # dep_sids is already a single constant stream id for every access
        # (stage_result_dependent tags everything "resgen_dep"), which by
        # itself forces full serialization regardless of `m` -- exactly the
        # dependent-chain behaviour we want to demonstrate.
        r_dep = mlp_model.simulate_mshr(dep_lats, dep_sids, m, l1_lat)
        r_indep = mlp_model.simulate_mshr(indep_lats, indep_sids, m, l1_lat)
        rows.append({"mshr": m, "variant": "dependent_chain", **r_dep})
        rows.append({"mshr": m, "variant": "independent_streams", **r_indep})
        print(f"    MSHR={m:>3d}  dependent={r_dep['total_cycles']:>10,d} cyc   "
              f"independent={r_indep['total_cycles']:>10,d} cyc")

    return rows


# ============================================================================
# 8. Experiment G -- final combined recommendation
# ============================================================================

def exp_g_final_recommendation(best_prefetcher_label: str, best_prefetcher_factory,
                               best_vc: int):
    print("\n=== Exp G: final configuration comparison ===")
    configs = [
        ("A_baseline", dict()),
        ("B_victim_only", dict(victim_entries=best_vc)),
        ("C_prefetch_only", dict(prefetcher=best_prefetcher_factory())),
        ("D_victim+prefetch (recommended)", dict(victim_entries=best_vc, prefetcher=
     best_prefetcher_factory())),
    ]
    rows = []
    for label, kw in configs:
        res = run_pipeline_staged(DEFAULT_CFG, resgen_variant="independent", **kw)
        s = dict(res["overall"])
        s["config"] = label
        rows.append(s)
        print(f"    {label:<34s} clock={s['clock']:>10,d}  AMAT={s['amat']:6.2f}  "
              f"L1miss%={s['l1_miss_pct']:5.2f}  dram_acc={s['dram_accesses']:>7,d}")
    return rows


# ============================================================================
# 9. Output: CSV / JSON / master ledger / LaTeX tables / plots
# ============================================================================

def write_csv(path: Path, rows, fieldnames=None) -> None:
    if not rows:
        return
    if fieldnames is None:
        fieldnames = list(rows[0].keys())
    with open(path, "w", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fieldnames, extrasaction="ignore")
        w.writeheader()
        for row in rows:
            w.writerow(row)


def append_master_ledger(entries) -> None:
    path = RESULTS_DIR / "master_ledger.csv"
    fields = ["timestamp", "experiment", "variant", "clock", "amat", "l1_miss_pct",
              "l2_miss_pct", "llc_miss_pct", "dram_accesses", "notes"]
    write_header = not path.exists()
    ts = time.strftime("%Y-%m-%d %H:%M:%S")
    with open(path, "a", newline="") as f:
        w = csv.DictWriter(f, fieldnames=fields)
        if write_header:
            w.writeheader()
        for e in entries:
            row = {"timestamp": ts, **e}
            w.writerow({k: row.get(k, "") for k in fields})


def main() -> None:
    RESULTS_DIR.mkdir(parents=True, exist_ok=True)
```

```
    PLOTS_DIR.mkdir(parents=True, exist_ok=True)
    t_start = time.time()

    res_a = exp_a_baseline()
    write_csv(RESULTS_DIR / "expA_stages.csv", res_a["stages"])
    write_csv(RESULTS_DIR / "expA_cold_steady.csv", res_a["cold_steady"])
    with open(RESULTS_DIR / "expA_overall.json", "w") as f:
        json.dump(res_a["overall"], f, indent=2)

    res_b = exp_b_victim_cache()
    write_csv(RESULTS_DIR / "expB_victim_sweep.csv", res_b["vc_sweep"])
    with open(RESULTS_DIR / "expB_assoc_compare.json", "w") as f:
        json.dump(res_b["assoc_compare"], f, indent=2)

    res_b2 = exp_b_conflict_microbench()
    write_csv(RESULTS_DIR / "expB_conflict_microbench.csv", res_b2)

    res_c = exp_c_prefetchers()
    write_csv(RESULTS_DIR / "expC_prefetchers.csv", res_c)

    res_d = exp_d_distance_sweep()
    write_csv(RESULTS_DIR / "expD_distance_sweep.csv", res_d)

    res_e = exp_e_pollution_thrashing()
    write_csv(RESULTS_DIR / "expE_pressure_baseline.csv", res_e["baseline"])
    write_csv(RESULTS_DIR / "expE_pressure_mitigation.csv", res_e["mitigation"])

    res_f = exp_f_mshr()
    write_csv(RESULTS_DIR / "expF_mshr_sweep.csv", res_f)

    # pick the best prefetcher from Exp C by lowest overall clock (excluding "none")
    best_c = min((r for r in res_c if r["prefetcher"] != "none"), key=lambda r: r["clock"])
    best_label = best_c["prefetcher"]
    factory_map = dict(PREFETCH_FACTORIES)
    best_vc = min(res_b["vc_sweep"][1:], key=lambda r: r["clock"])["victim_entries"]
    print(f"\n[selection] best single prefetcher from Exp C: {best_label}   best VC size from Exp B: {
     best_vc}")

    res_g = exp_g_final_recommendation(best_label, factory_map[best_label], best_vc)
    write_csv(RESULTS_DIR / "expG_final_comparison.csv", res_g)

    ledger_entries = []
    for row in res_a["stages"]:
        ledger_entries.append({"experiment": "A_baseline", "variant": row["stage"], **row,
                               "notes": "baseline characterization"})
    for row in res_b["vc_sweep"]:
        ledger_entries.append({"experiment": "B_victim_cache", "variant": f"vc{row['victim_entries']}",
                               **row, "notes": "victim cache sweep"})
    for row in res_c:
        ledger_entries.append({"experiment": "C_prefetch", "variant": row["prefetcher"], **row,
                               "notes": "prefetcher comparison"})
    for row in res_d:
        ledger_entries.append({"experiment": "D_distance", "variant": f"{row['kind']}_d{row['distance
     ']}",
                               **row, "notes": "prefetch distance sweep"})
    for row in res_g:
        ledger_entries.append({"experiment": "G_final", "variant": row["config"], **row,
                               "notes": "final recommendation comparison"})
    append_master_ledger(ledger_entries)

    write_latex_tables(res_a, res_b, res_b2, res_c, res_d, res_e, res_f, res_g, best_label, best_vc)
    make_plots(res_a, res_b, res_b2, res_c, res_d, res_e, res_f, res_g)

    with open(RESULTS_DIR / "raw_results.json", "w") as f:
        json.dump({
            "exp_a": {"stages": res_a["stages"], "cold_steady": res_a["cold_steady"], "overall": res_a[
     "overall"]},
            "exp_b": res_b,
            "exp_b_conflict_microbench": res_b2,
            "exp_c": res_c,
            "exp_d": res_d,
            "exp_e": res_e,
            "exp_f": res_f,
```

```
            "exp_g": res_g,
            "selection": {"best_prefetcher": best_label, "best_vc": best_vc},
        }, f, indent=2, default=str)

    print(f"\nAll done in {time.time()-t_start:.1f}s. Results written to: {RESULTS_DIR}")


# ============================================================================
# 10. LaTeX table generation
# ============================================================================

def esc(s):
    return str(s).replace("_", r"\_")


def write_latex_tables(res_a, res_b, res_b2, res_c, res_d, res_e, res_f, res_g, best_label, best_vc) ->
      None:
    L = []

    L.append("%%SECTION:A%%")
    # --- Table: Exp A stage-wise baseline ---
    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{lrrrrrr}\toprule")
    L.append(r"Stage & Accesses & L1 Miss\% & L2 Miss\% & LLC Miss\% & DRAM Acc. & AMAT \\ \midrule")
    for row in res_a["stages"]:
        L.append(f"{esc(row['stage'])} & {row['total_accesses']:,} & {row['l1_miss_pct']:.2f} & "
                 f"{row['l2_miss_pct']:.2f} & {row['llc_miss_pct']:.2f} & {row['dram_accesses']:,} & "
                 f"{row['amat']:.2f} \\\\")
    L.append(r"\bottomrule\end{tabular}")
    L.append(r"\caption{Exp A: baseline per-stage characterization (no victim cache, no prefetching).}"
     )
    L.append(r"\label{tab:expA}\end{table}")
    L.append("")

    # --- Table: Exp A cold vs steady ---
    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{lrrr}\toprule")
    L.append(r"Stage segment & Accesses & L1 Miss\% & AMAT \\ \midrule")
    for row in res_a["cold_steady"]:
        L.append(f"{esc(row['stage'])} & {row['total_accesses']:,} & {row['l1_miss_pct']:.2f} & {row['
     amat']:.2f} \\\\")
    L.append(r"\bottomrule\end{tabular}")
    L.append(r"\caption{Exp A: first 20\% (cold) vs. remaining 80\% (steady-state) of each stage.}")
    L.append(r"\label{tab:expA-cold}\end{table}")
    L.append("")

    L.append("%%SECTION:B%%")
    # --- Table: Exp B victim cache ---
    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{rrrrrr}\toprule")
    L.append(r"VC entries & Extra bytes & Clock (cyc) & L1 Miss\% & VC hits & AMAT \\ \midrule")
    for row in res_b["vc_sweep"]:
        L.append(f"{row['victim_entries']} & {row['extra_storage_bytes']:,} & {row['clock']:,} & "
                 f"{row['l1_miss_pct']:.2f} & {row['vc_hits']:,} & {row['amat']:.2f} \\\\")
    a = res_b["assoc_compare"]
    L.append(rf"\midrule 13-way L1 (+4KB, no VC) & 4{{,}}096 & {a['clock']:,} & {a['l1_miss_pct']:.2f}
     & -- & {a['amat']:.2f} \\")
    L.append(r"\bottomrule\end{tabular}")
    L.append(r"\caption{Exp B: victim-cache size sweep vs. an equal-extra-storage associativity
     increase.}")
    L.append(r"\label{tab:expB}\end{table}")
    L.append("")

    L.append("%%SECTION:Bconflict%%")
    # --- Table: Exp B supplementary conflict microbenchmark ---
    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{rrrr}\toprule")
    L.append(r"VC entries & L1 Miss\% & Clock (cyc) & VC hits \\ \midrule")
    for row in res_b2:
        L.append(f"{row['victim_entries']} & {row['l1_miss_pct']:.2f} & {row['clock']:,} & {row['
     vc_hits']:,} \\\\")
    L.append(r"\bottomrule\end{tabular}")
    n_lines = res_b2[0]["N_lines"]; ways = res_b2[0]["l1_ways"]
```

```
    L.append(rf"\caption{{Exp B supplementary: {n_lines} addresses round-robin thrashing one {ways}-way L1 set.}}")
    L.append(r"\label{tab:expB-conflict}\end{table}")
    L.append("")

    L.append("%%SECTION:C%%")
    # --- Table: Exp C prefetchers ---
    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{lrrrrrr}\toprule")
    L.append(r"Prefetcher & Clock (cyc) & L1 Miss\% & Accuracy\% & Coverage\% & Timeliness\% & Issued \\ \midrule")
    for row in res_c:
        L.append(f"{esc(row['prefetcher'])} & {row['clock']:,} & {row['l1_miss_pct']:.2f} & "
                 f"{row['prefetch_accuracy_pct']:.1f} & {row['coverage_pct']:.1f} & "
                 f"{row['prefetch_timeliness_pct']:.1f} & {row['prefetch_issued']:,} \\\\")
    L.append(r"\bottomrule\end{tabular}")
    L.append(r"\caption{Exp C: prefetch mechanism comparison, full pipeline.}")
    L.append(r"\label{tab:expC}\end{table}")
    L.append("")

    L.append("%%SECTION:D%%")
    # --- Table: Exp D distance sweep (condensed: min/max per kind) ---
    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{lrrrrr}\toprule")
    L.append(r"Kind & Distance & Clock (cyc) & L1 Miss\% & Wasted-evicted PF & Useful-data evictions \\ \midrule")
    for row in res_d:
        L.append(f"{esc(row['kind'])} & {row['distance']} & {row['clock']:,} & {row['l1_miss_pct']:.2f} & "
                 f"{row['prefetch_wasted_evicted']:,} & {row['useful_data_evictions']:,} \\\\")
    L.append(r"\bottomrule\end{tabular}")
    L.append(r"\caption{Exp D: prefetch distance sweep, full pipeline (distance 0 = no prefetch).}")
    L.append(r"\label{tab:expD}\end{table}")
    L.append("")

    L.append("%%SECTION:E%%")
    # --- Table: Exp E pollution/thrashing ---
    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{rrr}\toprule")
    L.append(r"Pressure level & Aggregation-state L1 hit\% & Overall L1 Miss\% \\ \midrule")
    for row in res_e["baseline"]:
        L.append(f"{row['pressure']} & {row['agg_state_hit_pct']:.2f} & {row['l1_miss_pct']:.2f} \\\\")
    L.append(r"\bottomrule\end{tabular}")
    L.append(r"\caption{Exp E: baseline aggregation-state residency under rising interleave pressure.}")
    L.append(r"\label{tab:expE-baseline}\end{table}")
    L.append("")

    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{rlrr}\toprule")
    L.append(r"Pressure & Mitigation & Aggregation-state L1 hit\% & Overall L1 Miss\% \\ \midrule")
    for row in res_e["mitigation"]:
        L.append(f"{row['pressure']} & {esc(row['config'])} & {row['agg_state_hit_pct']:.2f} & {row['l1_miss_pct']:.2f} \\\\")
    L.append(r"\bottomrule\end{tabular}")
    L.append(r"\caption{Exp E: does the victim cache and/or prefetching alleviate high-pressure thrashing?}")
    L.append(r"\label{tab:expE-mitigation}\end{table}")
    L.append("")

    L.append("%%SECTION:F%%")
    # --- Table: Exp F MSHR sweep ---
    L.append(r"\begin{table}[H]\centering\small")
    L.append(r"\begin{tabular}{rrr}\toprule")
    L.append(r"MSHRs & Dependent chain (cyc) & Independent streams (cyc) \\ \midrule")
    dep = {r["mshr"]: r for r in res_f if r["variant"] == "dependent_chain"}
    ind = {r["mshr"]: r for r in res_f if r["variant"] == "independent_streams"}
    for m in sorted(dep):
        L.append(f"{m} & {dep[m]['total_cycles']:,} & {ind[m]['total_cycles']:,} \\\\")
    L.append(r"\bottomrule\end{tabular}")
    L.append(r"\caption{Exp F: total Result-Generation execution cycles vs. MSHR count.}")
    L.append(r"\label{tab:expF}\end{table}")
    L.append("")
```

```
665
666     L.append("%%SECTION:G%%")
667     # --- Table: Exp G final ---
668     L.append(r"\begin{table}[H]\centering\small")
669     L.append(r"\begin{tabular}{lrrrr}\toprule")
670     L.append(r"Configuration & Clock (cyc) & AMAT & L1 Miss\% & DRAM Acc. \\ \midrule")
671     for row in res_g:
672         L.append(f"{esc(row['config'])} & {row['clock']:,} & {row['amat']:.2f} & {row['l1_miss_pct']:.2
      f} & "
673                  f"{row['dram_accesses']:,} \\\\")
674     L.append(r"\bottomrule\end{tabular}")
675     L.append(rf"\caption{{Exp G: final configuration comparison (selected prefetcher: {esc(best_label)
      }, "
676              rf"selected victim-cache size: {best_vc} entries).}}")
677     L.append(r"\label{tab:expG}\end{table}")
678
679     with open(RESULTS_DIR / "tables.tex", "w") as f:
680         f.write("\n".join(x for x in L if not x.startswith("%%SECTION:")) + "\n")
681
682     tables_dir = RESULTS_DIR / "tables"
683     tables_dir.mkdir(parents=True, exist_ok=True)
684     current, buf = None, []
685     for line in L:
686         if line.startswith("%%SECTION:"):
687             if current is not None:
688                 with open(tables_dir / f"tables_{current}.tex", "w") as f:
689                     f.write("\n".join(buf) + "\n")
690             current = line.split(":")[1].rstrip("%")
691             buf = []
692         else:
693             buf.append(line)
694     if current is not None:
695         with open(tables_dir / f"tables_{current}.tex", "w") as f:
696             f.write("\n".join(buf) + "\n")
697
698
699 # ============================================================================
700 # 11. Plots
701 # ============================================================================
702
703 def make_plots(res_a, res_b, res_b2, res_c, res_d, res_e, res_f, res_g) -> None:
704     # Exp A: per-stage miss rate by level
705     stages = [r["stage"] for r in res_a["stages"]]
706     fig, ax = plt.subplots(figsize=(9, 4.5))
707     x = range(len(stages))
708     w = 0.25
709     ax.bar([i - w for i in x], [r["l1_miss_pct"] for r in res_a["stages"]], w, label="L1")
710     ax.bar(list(x), [r["l2_miss_pct"] for r in res_a["stages"]], w, label="L2")
711     ax.bar([i + w for i in x], [r["llc_miss_pct"] for r in res_a["stages"]], w, label="LLC")
712     ax.set_xticks(list(x)); ax.set_xticklabels(stages, rotation=20, ha="right")
713     ax.set_ylabel("Miss rate (%)"); ax.set_title("Exp A: baseline per-stage miss rate")
714     ax.legend(); fig.tight_layout()
715     fig.savefig(PLOTS_DIR / "expA_stage_miss_rates.png", dpi=150); plt.close(fig)
716
717     # Exp B: victim cache
718     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4.2))
719     vc_sizes = [r["victim_entries"] for r in res_b["vc_sweep"]]
720     ax1.plot(vc_sizes, [r["clock"] for r in res_b["vc_sweep"]], marker="o")
721     ax1.axhline(res_b["assoc_compare"]["clock"], color="tab:red", linestyle="--",
722                 label="13-way L1 (+4KB, no VC)")
723     ax1.set_xlabel("Victim cache entries"); ax1.set_ylabel("Total clock (cycles)")
724     ax1.set_title("Execution cycles vs. VC size"); ax1.legend()
725     ax2.plot(vc_sizes, [r["l1_miss_pct"] for r in res_b["vc_sweep"]], marker="o", color="tab:green")
726     ax2.set_xlabel("Victim cache entries"); ax2.set_ylabel("L1 miss rate (%)")
727     ax2.set_title("L1 miss rate vs. VC size")
728     fig.tight_layout(); fig.savefig(PLOTS_DIR / "expB_victim_cache.png", dpi=150); plt.close(fig)
729
730     # Exp B supplementary: conflict microbenchmark. L1 miss% is flat at
731     # 100% by definition (a victim-cache hit is still an L1 miss) even
732     # though the victim cache is doing all the work here, so plot the
733     # metrics that actually show it: clock and victim-cache hits.
734     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4.2))
735     vcs = [r["victim_entries"] for r in res_b2]
```

```
    ax1.plot(vcs, [r["clock"] for r in res_b2], marker="o", color="tab:red")
    ax1.set_xlabel("Victim cache entries"); ax1.set_ylabel("Total clock (cycles)")
    ax1.set_title("Execution cycles vs. VC size")
    ax2.plot(vcs, [r["vc_hits"] for r in res_b2], marker="s", color="tab:blue")
    ax2.set_xlabel("Victim cache entries"); ax2.set_ylabel("Victim-cache hits (count)")
    ax2.set_title("Recovered accesses vs. VC size")
    fig.suptitle(f"Exp B supplementary: {res_b2[0]['N_lines']}-line thrash on one "
                 f"{res_b2[0]['l1_ways']}-way set (L1 miss rate stays 100% throughout "
                 f"by definition -- see clock instead)", fontsize=10)
    fig.tight_layout(); fig.savefig(PLOTS_DIR / "expB_conflict_microbench.png", dpi=150); plt.close(fig)

    # Exp C: prefetcher comparison
    labels = [r["prefetcher"] for r in res_c]
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.5))
    ax1.bar(labels, [r["clock"] for r in res_c], color="tab:blue")
    ax1.set_ylabel("Total clock (cycles)"); ax1.set_title("Exp C: execution cycles by prefetcher")
    ax1.tick_params(axis="x", rotation=30)
    xi = range(len(labels))
    ax2.bar([i - 0.2 for i in xi], [r["prefetch_accuracy_pct"] for r in res_c], 0.2, label="Accuracy")
    ax2.bar(list(xi), [r["coverage_pct"] for r in res_c], 0.2, label="Coverage")
    ax2.bar([i + 0.2 for i in xi], [r["prefetch_timeliness_pct"] for r in res_c], 0.2, label="
    Timeliness")
    ax2.set_xticks(list(xi)); ax2.set_xticklabels(labels, rotation=30, ha="right")
    ax2.set_ylabel("%"); ax2.set_title("Accuracy / coverage / timeliness"); ax2.legend(fontsize=8)
    fig.tight_layout(); fig.savefig(PLOTS_DIR / "expC_prefetchers.png", dpi=150); plt.close(fig)

    # Exp D: distance sweep
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(11, 4.5))
    none_clock = next(r["clock"] for r in res_d if r["kind"] == "none")
    for kind, marker in (("stride", "o"), ("stream", "s")):
        pts = [r for r in res_d if r["kind"] == kind]
        ax1.plot([r["distance"] for r in pts], [r["clock"] for r in pts], marker=marker, label=kind)
    ax1.axhline(none_clock, color="gray", linestyle="--", label="no prefetch")
    ax1.set_xlabel("Prefetch distance"); ax1.set_ylabel("Total clock (cycles)")
    ax1.set_title("Exp D: execution cycles vs. distance"); ax1.legend(); ax1.set_xscale("log", base=2)
    for kind, marker in (("stride", "o"), ("stream", "s")):
        pts = [r for r in res_d if r["kind"] == kind]
        ax2.plot([r["distance"] for r in pts], [r["useful_data_evictions"] for r in pts],
                 marker=marker, label=f"{kind}: useful-data evictions")
    ax2.set_xlabel("Prefetch distance"); ax2.set_ylabel("Useful-data evictions (count)")
    ax2.set_title("Pollution vs. distance"); ax2.legend(fontsize=8); ax2.set_xscale("log", base=2)
    fig.tight_layout(); fig.savefig(PLOTS_DIR / "expD_distance_sweep.png", dpi=150); plt.close(fig)

    # Exp E: pollution/thrashing
    fig, ax = plt.subplots(figsize=(7, 4.5))
    ax.plot([r["pressure"] for r in res_e["baseline"]], [r["agg_state_hit_pct"] for r in res_e["
    baseline"]],
            marker="o", label="baseline")
    for label in sorted(set(r["config"] for r in res_e["mitigation"])):
        pts = [r for r in res_e["mitigation"] if r["config"] == label]
        ax.plot([r["pressure"] for r in pts], [r["agg_state_hit_pct"] for r in pts],
                marker="s", linestyle="--", label=label)
    ax.set_xlabel("Pressure level (extraction stride / injected traffic)")
    ax.set_ylabel("Aggregation-state L1 hit rate (%)")
    ax.set_title("Exp E: hot aggregation-state residency under rising pressure")
    ax.legend(fontsize=8); ax.set_xscale("log", base=2)
    fig.tight_layout(); fig.savefig(PLOTS_DIR / "expE_pollution_thrashing.png", dpi=150); plt.close(fig)

    # Exp F: MSHR sweep
    fig, ax = plt.subplots(figsize=(7, 4.5))
    dep = {r["mshr"]: r["total_cycles"] for r in res_f if r["variant"] == "dependent_chain"}
    ind = {r["mshr"]: r["total_cycles"] for r in res_f if r["variant"] == "independent_streams"}
    xs = sorted(dep)
    ax.plot(xs, [dep[m] for m in xs], marker="o", label="dependent chain")
    ax.plot(xs, [ind[m] for m in xs], marker="s", label="independent streams (K=8)")
    ax.axvline(8, color="gray", linestyle=":", label="K = 8 streams")
    ax.set_xlabel("MSHR count (outstanding misses)"); ax.set_ylabel("Total cycles (Result Generation)")
    ax.set_title("Exp F: blocking vs. non-blocking cache"); ax.legend(); ax.set_xscale("log", base=2)
    fig.tight_layout(); fig.savefig(PLOTS_DIR / "expF_mshr_sweep.png", dpi=150); plt.close(fig)

    # Exp G: final comparison
```

```
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 4.2))
    labels = [r["config"] for r in res_g]
    ax1.bar(labels, [r["clock"] for r in res_g], color="tab:purple")
    ax1.set_ylabel("Total clock (cycles)"); ax1.set_title("Exp G: final configuration clock")
    ax1.tick_params(axis="x", rotation=20)
    ax2.bar(labels, [r["l1_miss_pct"] for r in res_g], color="tab:orange")
    ax2.set_ylabel("L1 miss rate (%)"); ax2.set_title("Exp G: final configuration L1 miss rate")
    ax2.tick_params(axis="x", rotation=20)
    fig.tight_layout(); fig.savefig(PLOTS_DIR / "expG_final.png", dpi=150); plt.close(fig)


if __name__ == "__main__":
    main()
```